S11BLH51	SOFTWARE ENGINEERING DESIGN AND DEVELOPMENT

COURSE OBJECTIVES

- To understand the fundamentals of software process and development, requirements engineering and analysis modelling.
- To use UML in requirements elicitation and designing.
- To understand concepts of relationships and aggregations and to test the software against its requirements specification.

LIST OF EXPERIMENTS

1(a). Study Experiment – Object Oriented Concepts
1(b). Study Aim and Scope of UML Diagrams –Diagrams
2. Create a system to perform book bank operation
3. Create a system to perform the Exam registration system.
4. Design an object-oriented model for Online Course reservation system.
5. Design an object-oriented model for Passport automation System.
6. Design an object-oriented model for ATM management system.
7. Design an object-oriented model for Hospital Management system.
8. Design an object-oriented model for Conference management System.

EX-NO-1(a)	Study Experiment – Object Oriented Concepts

Aim:

To study about the object-oriented concepts.

Description:

Basic OO Concepts

Introduction:

Goal of OO: Reduce complexity of software development by keeping details, and especially changes to details, from spreading throughout the entire program

This presentation assumes "Basic Class Design" presentation.

Definitions

- Client Code - the code that uses the classes under discussion.
- Coupling - code in one module depends on code in another module Change in one forces rewrite (horrible!), or recompile (annoying), of code in the other. Four key concepts of OOP

1. Abstraction - responsibilities (interface) is different from implementation
    - Distinguish between interface and its behavior vs. the implementation and its details
    - A class provides some services, takes on some responsibilities, that are defined by the public interface. How it works inside doesn't matter.
    - Client should be able to use just the public interface, and not care about the implementation.
2. Encapsulation - guarantee responsibilities by protecting implementation from interference
    - Developer of a class can guarantee behavior of a class only if the internals are protected from outside interference. Specifying private access for the internals puts a wall around the internals, making a clear distinction between which code the class developer is responsible for, and what the client developers are responsible for.

3. Inheritance - share interface and/or implementation in related classes

   - Express commonalities between classes of objects by arranging them into an inheritance hierarchy. Allows functionality shared between classes to be written/debugged/maintained in one place, the base class, and then reused in descendant classes (shared implementation).

   - More importantly, allows client code to interact with classes in the hierarchy through an interface specified in the base class of the hierarchy (shared interface).

4. Inheritance - share interface and/or implementation in related classes

   - Express commonalities between classes of objects by arranging them into an inheritance hierarchy. Allows functionality shared between classes to be written/debugged/maintained in one place, the base class, and then reused in descendant classes (shared implementation).

   - More importantly, allows client code to interact with classes in the hierarchy through an interface specified in the base class of the hierarchy (shared interface).

5. Polymorphism - decouple client from exact class structure
   - Allows client code to use classes in an class hierarchy in a way that depends only on the public interface of the base class. Derived classes will implement this interface differently, but in ways compatible with the shared public interface.

General Approach for Designing Classes:

Do the class design work at the level of the public interfaces, not the private implementations.

I. Don't get bogged down in implementation details like "I can do with this with a map container and a deque!"

II. Think only about what the class responsibilities are and what they do in their public interfaces:

   - ✓ Class X is responsible for …., class Y for ….

   - ✓ When an X object needs … it calls the public member function … of the appropriate Y object with … as parameters, which returns …

III. Try writing pseudo code just for the interactions between class objects through their public interfaces.

IV. Keep this up until you can't stand it anymore, then make implementation choices and write the code.

Continue design thinking until you have thought of at least two reasonable ways to solve each design problem.

I. "Reasonable" here means "not obviously stupid."
II. Don't just jump on the first design you think of and hack it out.

III. All designs are imperfect - they all involve trade-offs. They are good in some ways, bad in others.

IV. A good design is good in the most important ways, and bad in the less important ways.

V. But there might be more than one good design - just different in the specific tradeoffs.

VI. You can't make an intelligent choice if you have only thought of one design - there could be another, better, simpler one.

EX-NO-1(b)	Study of UML Diagrams – Aim and Scope of Diagrams

AIM:

General study of UML

DESCRIPTION:

Why is UML important?

Let's look at this question from the point of view of the construction trade. Architects design buildings. Builders use the designs to create buildings. The more complicated the building, the more critical the communication between architect and builder. Blueprints are the standard graphical language that both architects and builders must learn as part of their trade.

Writing software is not unlike constructing a building. The more complicated the underlying system, the more critical the communication among everyone involved in creating and deploying the software. In the past decade, the UML has emerged as the software blueprint language for analysts, designers, and programmers alike. It is now part of the software trade. The UML gives everyone from business analyst to designer to programmer a common vocabulary to talk about software design.

The UML is applicable to object-oriented problem solving. Anyone interested in learning UML must be familiar with the underlying tenet of object-oriented problem solving -- it all begins with the construction of a model. A model is an abstraction of the underlying problem. The domain is the actual world from which the problem comes. Models consist of objects that interact by sending each other messages. Think of an object as "alive." Objects have things they know (attributes) and things they can do (behaviors or operations). The values of an object's attributes determine its state.

Classes are the "blueprints" for objects. A class wraps attributes (data) and behaviors (methods or functions) into a single distinct entity. Objects are instances of classes.

UML NOTATION

- Unified Modeling Language.
- Set of notations and conventions used to describe and model an application.
- Universal language for modeling systems.
- Standard notation for OO modeling systems.
- Does not specify methodology to develop an application.

UML DIAGRAMS

The heart of object-oriented problem solving is the construction of a model. The model abstracts the essential details of the underlying problem from its usually complicated real world. Several modeling tools are wrapped under the heading of the UML™, which stands for Unified Modeling Language™. The purpose of this course is to present important highlights of the UML.

At the center of the UML are its nine kinds of modeling diagrams, which we describe here.

- Class Diagram
- Use Case Diagram
- Behavioral Diagram
  - ✓ Interaction Diagram
    - ▪ Sequence Diagram
    - ▪ Collaboration Diagram
  - ✓ State Chart Diagram
  - ✓ Activity Diagram
- Implementation Diagram
  - ✓ Component Diagram
  - ✓ Deployment Diagram

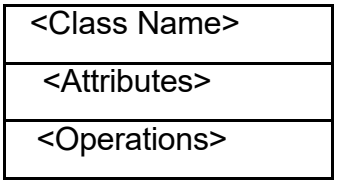
Some of the sections of this course contain links to pages with more detailed information. And every section has short questions. Use them to test your understanding of the section topic.

CLASS DIAGRAM

- Shows the static structure of the model.
- Collection of static modeling elements such as classes and their relationships connected as a graph.
- Provides visual representation of objects, relationships and their structures.

Class:-

- A class is a set of objects that share a common structure and common behavior.

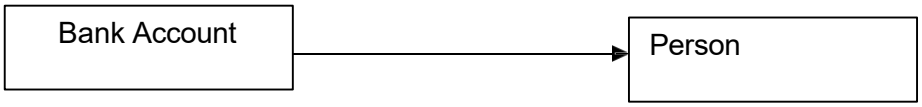


Interface:-

- Specifies the externally-visible operations of a class and/or component.

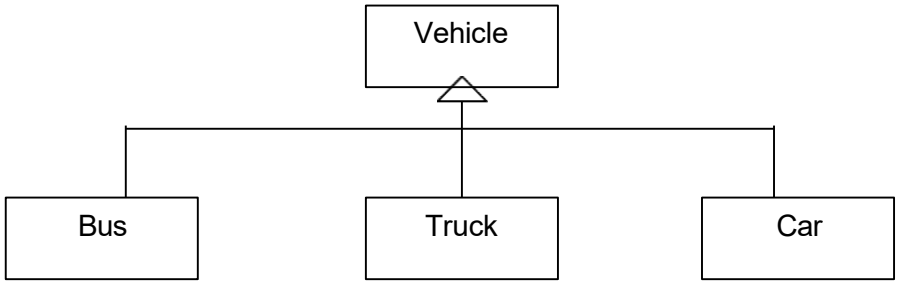
Association:-

- Model properties of associations.
- The properties are stored in a class and linked to the association relationship.
- Example,



Generalization:-

- A generalize relationship is a relationship between a more general class or use case and a more specific class or use case.
- Example,

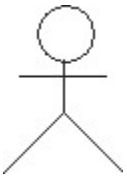


USE CASE DIAGRAM

- Set of use cases enclosed by system boundary, communication association between actors and use cases, and generalization among use cases.

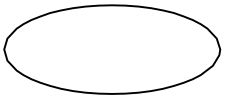
Actors:-

- External factors that interacts with the system from the user's perspective.



Use Cases:-

- Set of scenarios that describe how actor uses the system.
- Represented as,



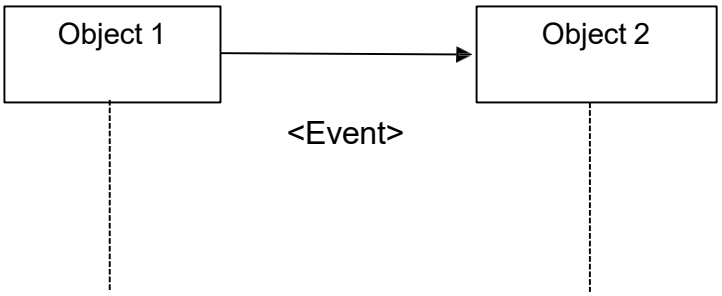
Relationship:-

- Communication – communications with the use case normally.
- Uses – Shown by generalization arrow from the use cases.
- Extends – Used when one case does more than another that is similar to it.

- Diagrams that describes how group of objects are collaborated.

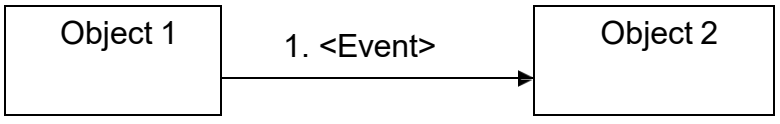
SEQUENCE DIAGRAM:

- Describes the behavior of the system through interaction between the system and the environment in time sequence.
- Two dimensions:
  - ✓ Vertical dimension – represents time.
  - ✓ Horizontal dimension – represents objects.
- Life line – Object's existence during the interaction.



COLLABORATION DIAGRAM:

- An interaction diagram that shows the order of messages that implement an operation or a transaction.
- Collaboration diagrams show objects, their links, and their messages.



Object:-

- An object has state, behavior, and identity.
- Objects interact through their links to other objects.

Link:-

- A link is an instance of an association, analogous to an object.

Message:-

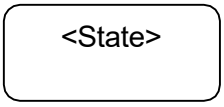
- A message is the communication carried between two objects that trigger an event.

STATECHART DIAGRAM

- Models the dynamic behavior of individual classes or any other kind of object.
- Shows the sequences of states, events, and actions.

State:-

- Represents a condition or situation during the life of an object during which it satisfies some condition or waits for some event.



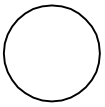
Start State:-

- Shows the beginning of workflow.



End state::-

- Represents the final or terminal state.

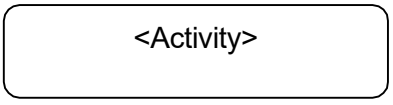


ACTIVITY DIAGRAM

- Used for modeling the sequence of activities in a process
- Special case of a state machine in which most of the states are activities and most of the transitions are implicitly triggered by completion of the actions in the source activities.

Activity:-

- Represents the performance of task or duty in a workflow.



Swim lanes:-

- Represents organizational units or roles within a business model.

IMPLEMENTATION DIAGRAM

- Shows the implementation phase of system development.
- Two types of implementation diagrams:
  - ✓ Component diagram
  - ✓ Deployment diagram

COMPONENT DIAGRAM

- Models the physical components in the design.

- A graph of the design's components connected by dependency relationships.
- Includes concept of packages.
- Package is used to show how classes are grouped together.

DEPLOYMENT DIAGRAM

- Shows the configuration of runtime processing elements and software components.
- It is a graph of nodes connected by communication association.
- Nodes are the components that are connected to other components through dependencies.
- Used in conjunction with component diagrams to show the distribution of physical modules.

EX-NO-2	BOOK BANK

AIM:

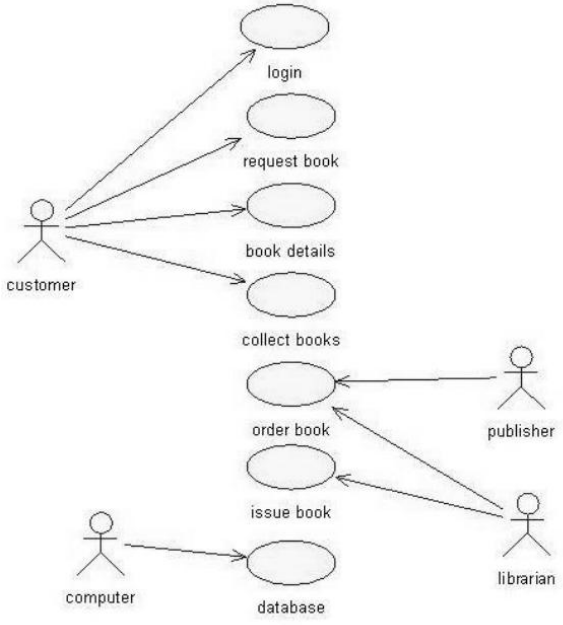
To create a system to perform book bank operation

PROBLEM STATEMENT:

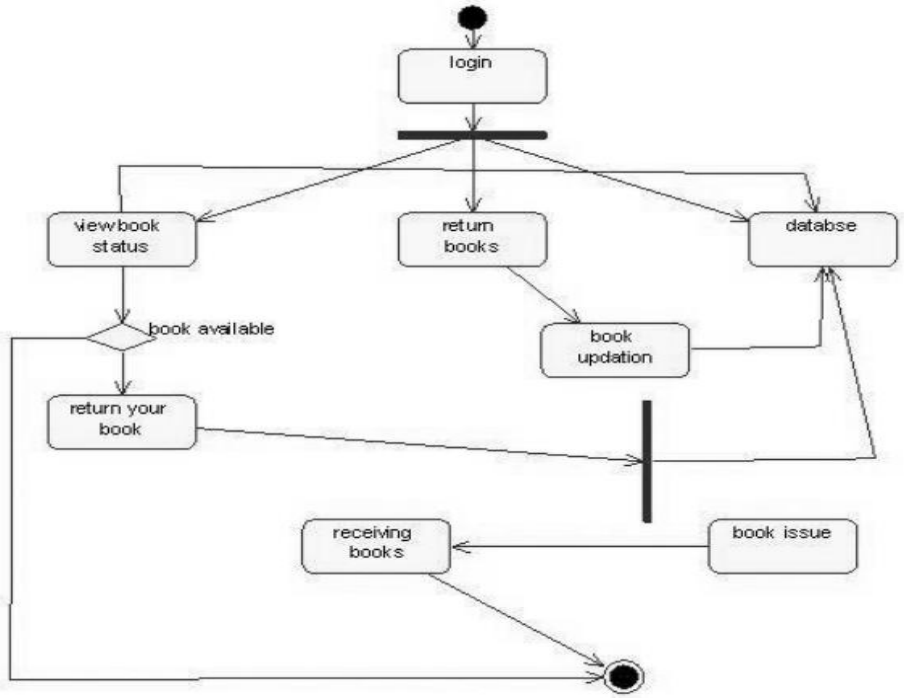
A Book Bank lends books and magazines to member, who is registered in the system. Also, it handles the purchase of new titles for the Book Bank. Popular titles are brought into multiple copies. Old books and magazines are removed when they are out or date or poor in condition. A member can reserve a book or magazine that is not currently available in the book bank, so that when it is returned or purchased by the book bank, that person is notified. The book bank can easily create, replace and delete information about the tiles, members, loans and reservations from the system.

USE-CASE DIAGRAM:

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled. It is represented using stick figure.

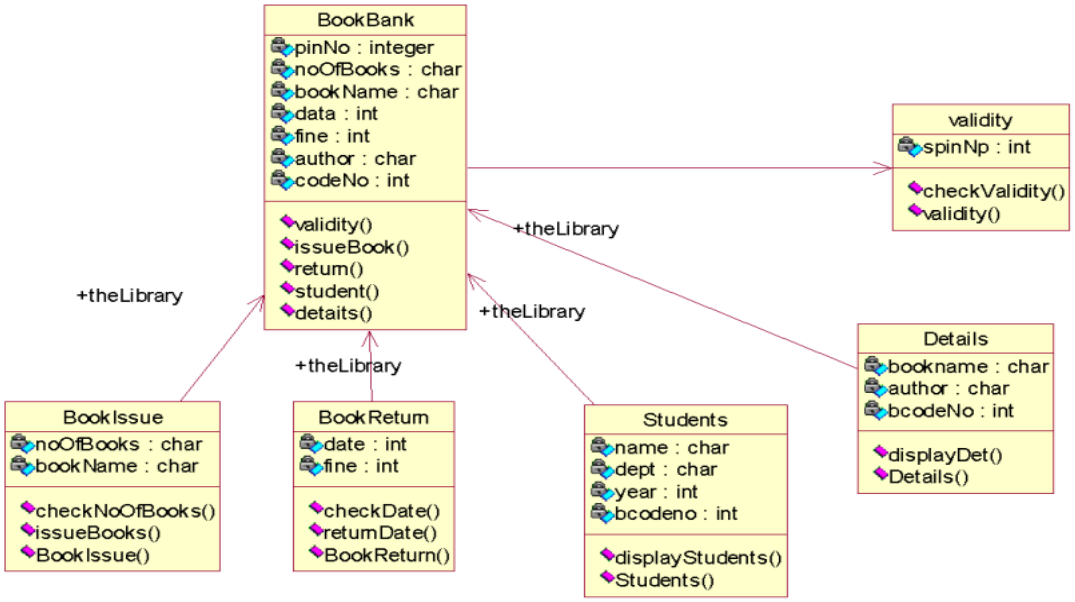


ACTIVITY DIAGRAM:



CLASS DIAGRAM:

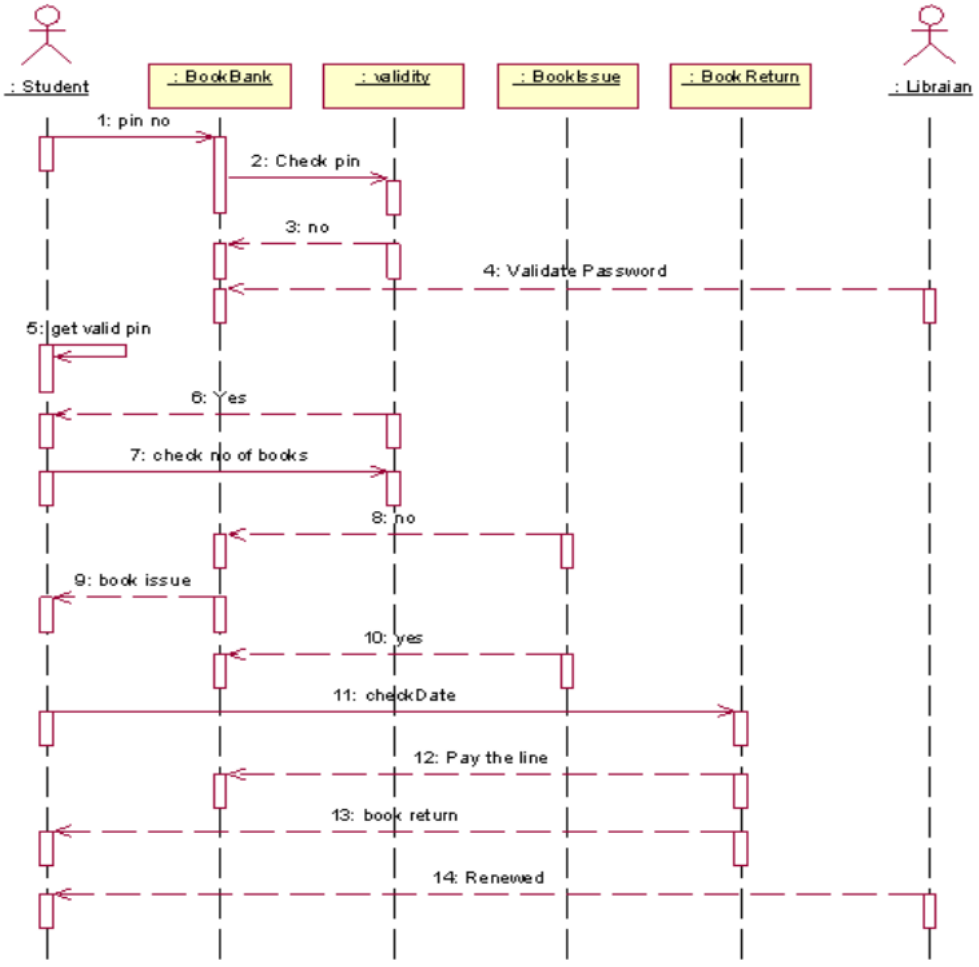
The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects



SEQUENCE DIAGRAM:

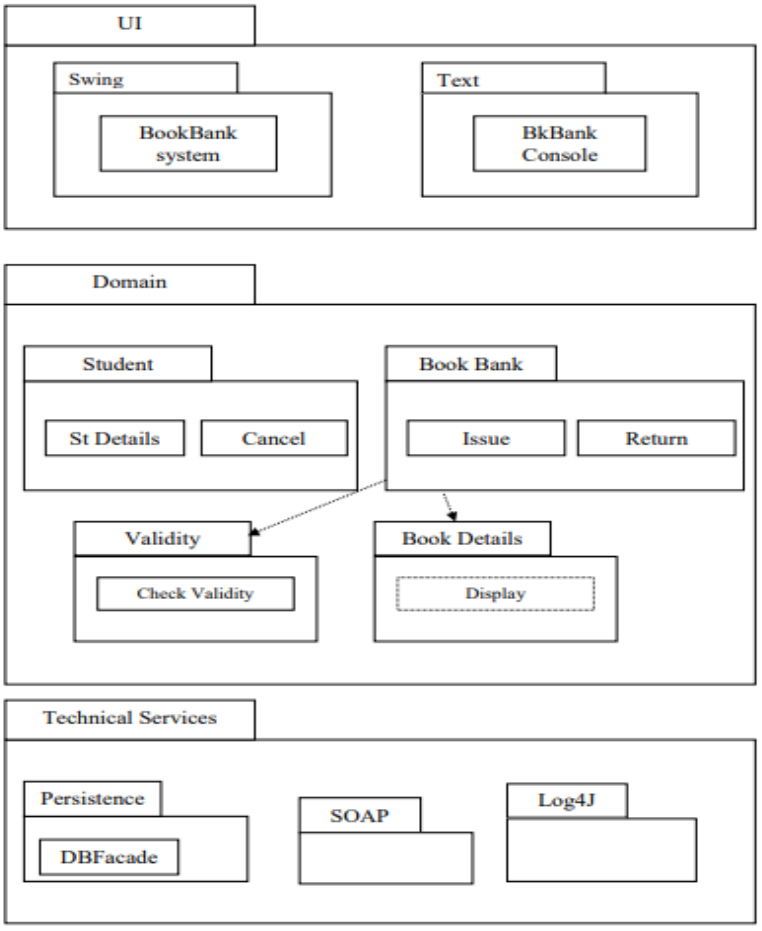
A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another, in which the “form” object is requesting an operation be performed by the “to” object. The “to” object performs the operation using a method that the class contains. It is also represented by the order in which things occur and how the objects in the system send message to one another.



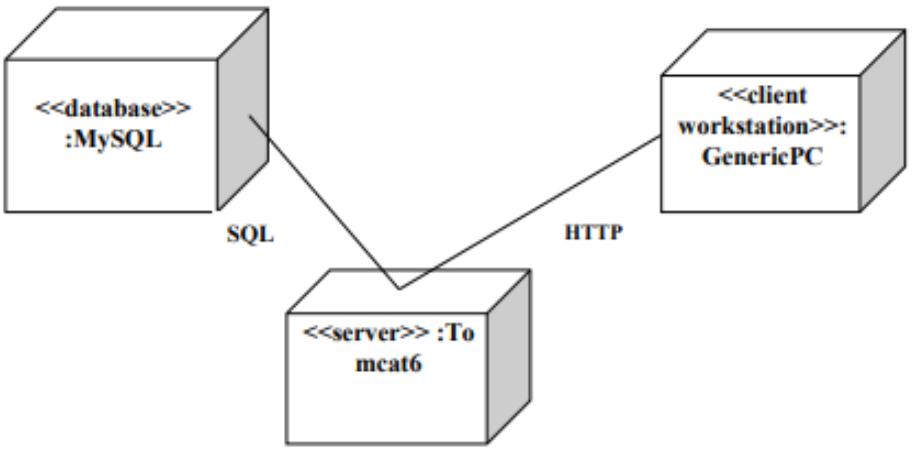
The diagrams show the pin no is entered and check the pin. Get no and validate password check the condition based on condition book issue and return are done. Pay the online and renewed

PARTIAL LAYERD LOGICAL ARCHITECTURE DIAGRAM:



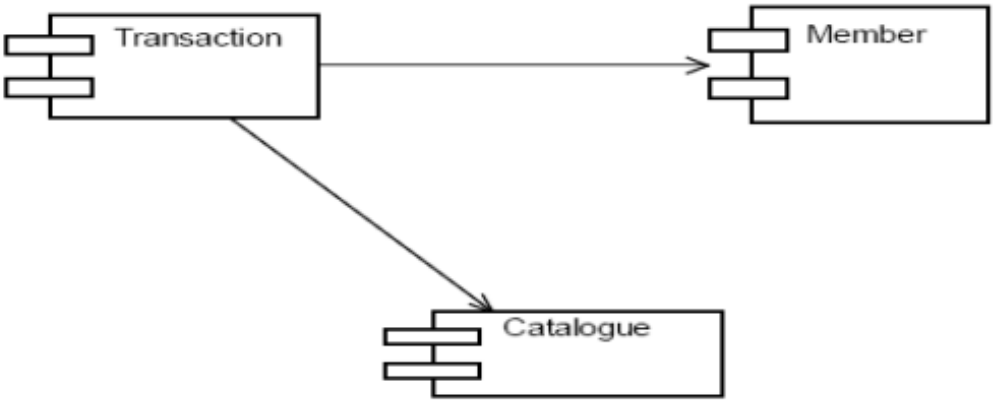
DEPLOYMENT DIAGRAM AND COMPONENT DIAGRAM

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed.



COMPONENT DIAGRAM

Component diagrams are used to visualize the organization and relationships



RESULT:

Thus, the Book Bank System has been successfully executed and generated.

EX-NO-3	EXAM REGISTRATION SYSTEM

AIM:

To create a system to perform the Exam Registration system

PROBLEM STATEMENT:

Exam Registration system.is used in the effective dispatch of registration form to all of the students. This system adopts a comprehensive approach to minimize the manual work and schedule resources, time in a cogent manner. The core of the system is to get the online registration form (with details such as name, reg.no etc.,) filled by the student whose testament is verified for its genuineness by the Exam Registration System with respect to the already existing information in the database.

USECASE DIAGRAM:

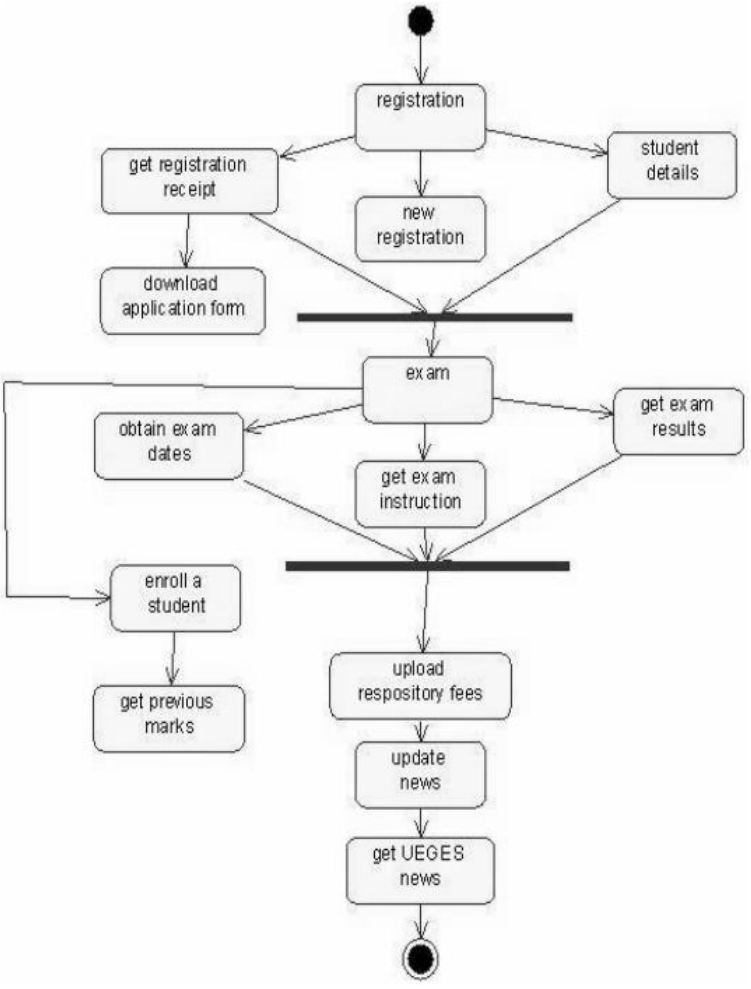
A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled. It is represented using stick figure.

The Exam Registration use cases in our system are:

1. Login
2. View exam details
3. View fees details
4. Pay fee
5. Display details
6. Logout

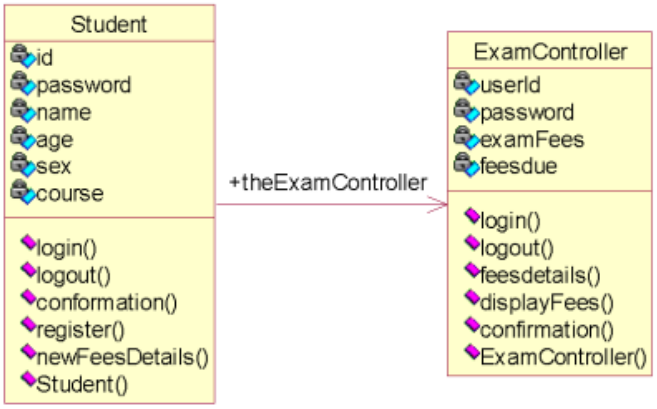


ACTIVITY DIAGRAM:



CLASS DIAGRAM:

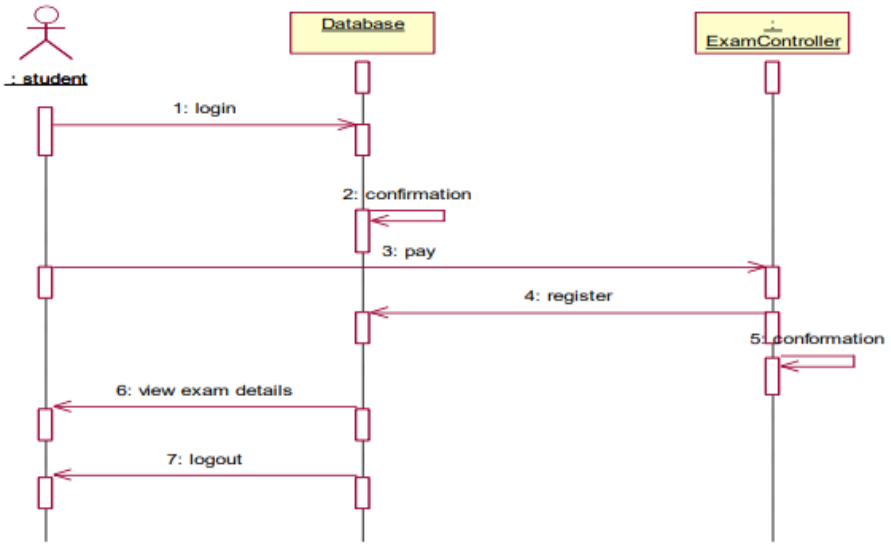
The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects.

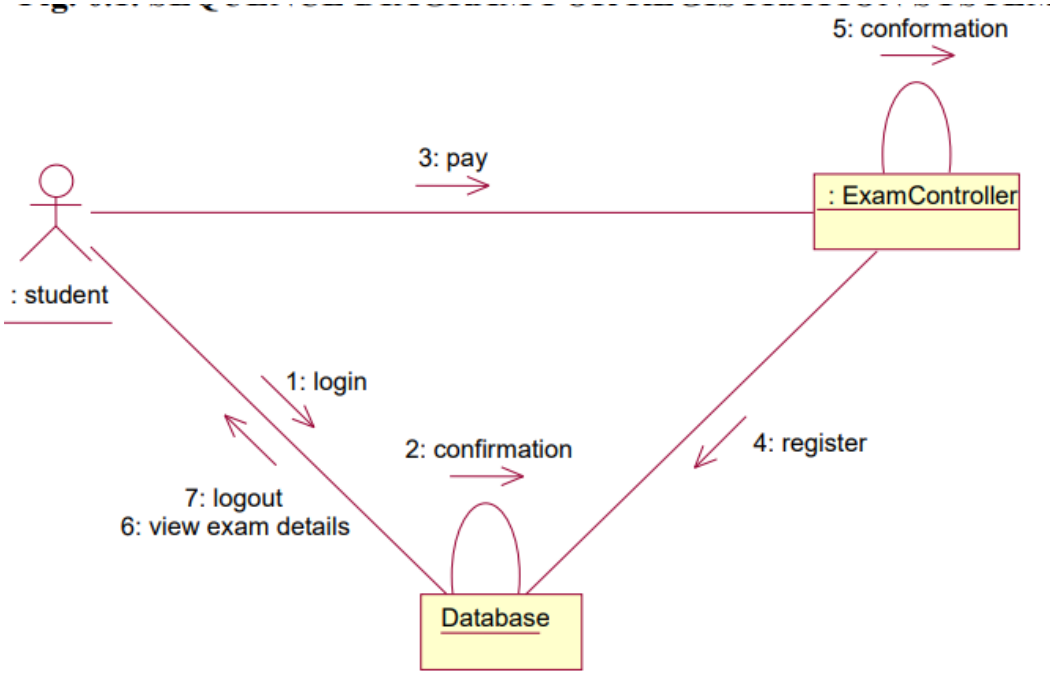


INTERACTION DIAGRAM:

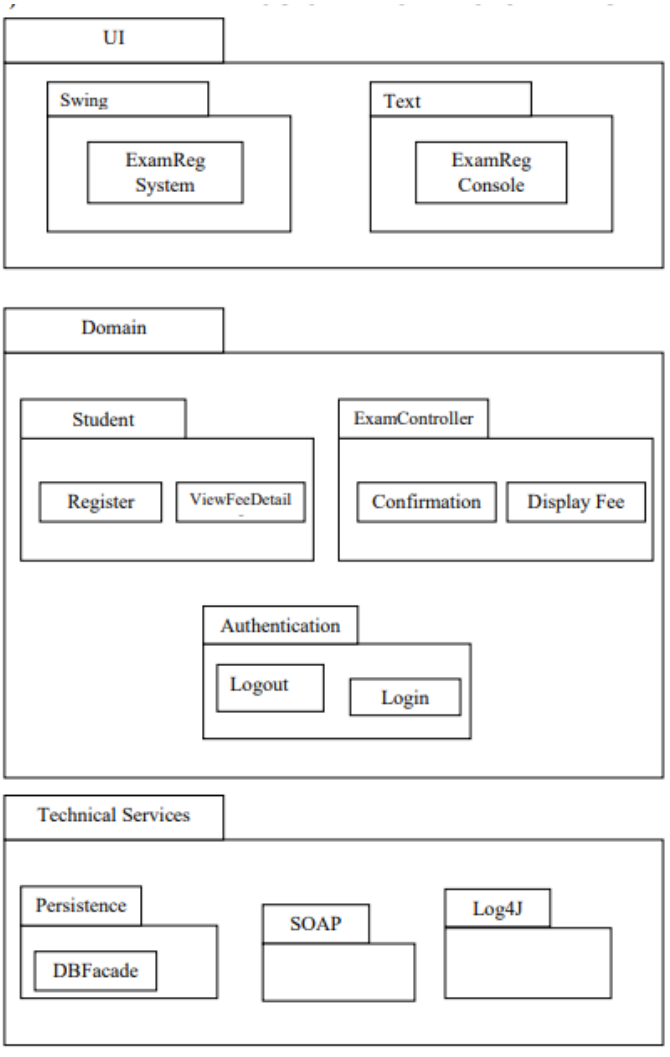
A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another



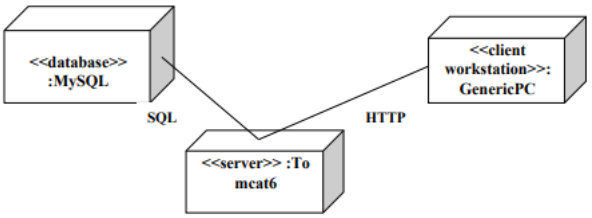


PARTIAL LAYERD LOGICAL ARCHITECTURE DIAGRAM:



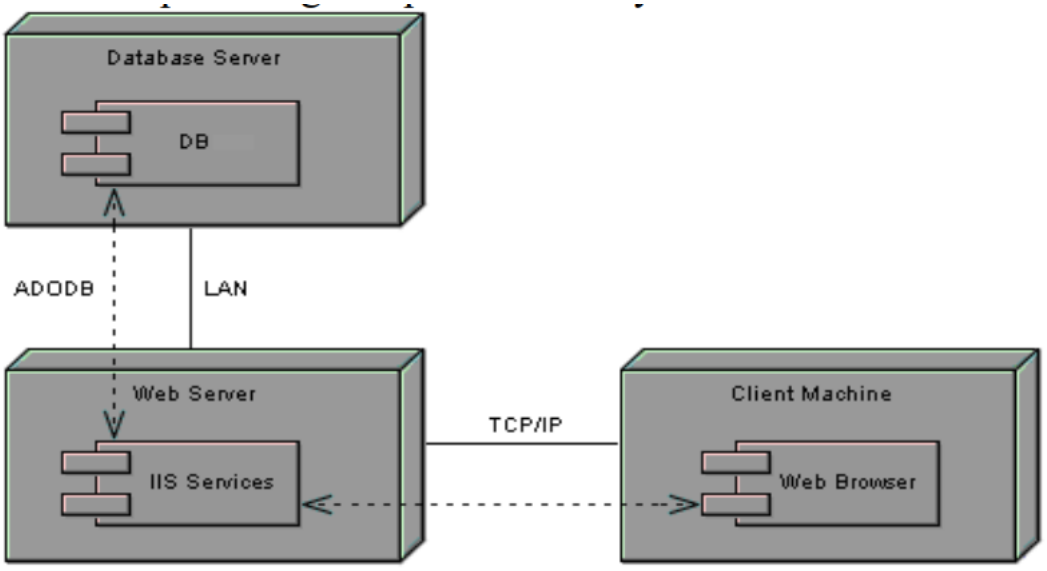
DEPLOYMENT DIAGRAM AND COMPONENT DIAGRAM

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed.



COMPONENT DIAGRAM

Component diagrams are used to visualize the organization and relationships among components in a system



RESULT:

Thus, the Exam Registration system has been successfully executed and generated.

EX.NO:4	Online Course Registration System

AIM: To develop the problem statement for Online Course Registration System.

PROBLEM STATEMENT: Online Course Registration System is meant for students who wish to apply for the course through online in a college. At first the system will list the available courses in the college. Then the student will check the available seats for the particular course in the college. If the interested course is available then the student will login and directed to fill the application. The system will check for the eligibility and if the student is eligible then they will direct for payment to reserve the course. If the course is reserved, then an acknowledgement will be sent to the student and also to the registrar of the college.

Software Requirements Specification for ONLINE COURSE REGISTRATION

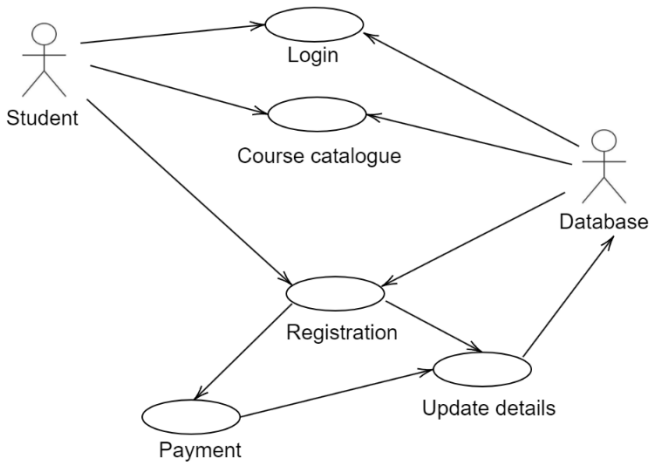
1. Introduction

1.1 Purpose

This SRS is developed for the entire system. Course Registration by means of Direct communication would consume time and takes more man power. In order to resolve this one, an Automated Online Course Registration System provides a smart way to reserve the course through online which helps the student to apply in an efficient and convenient way

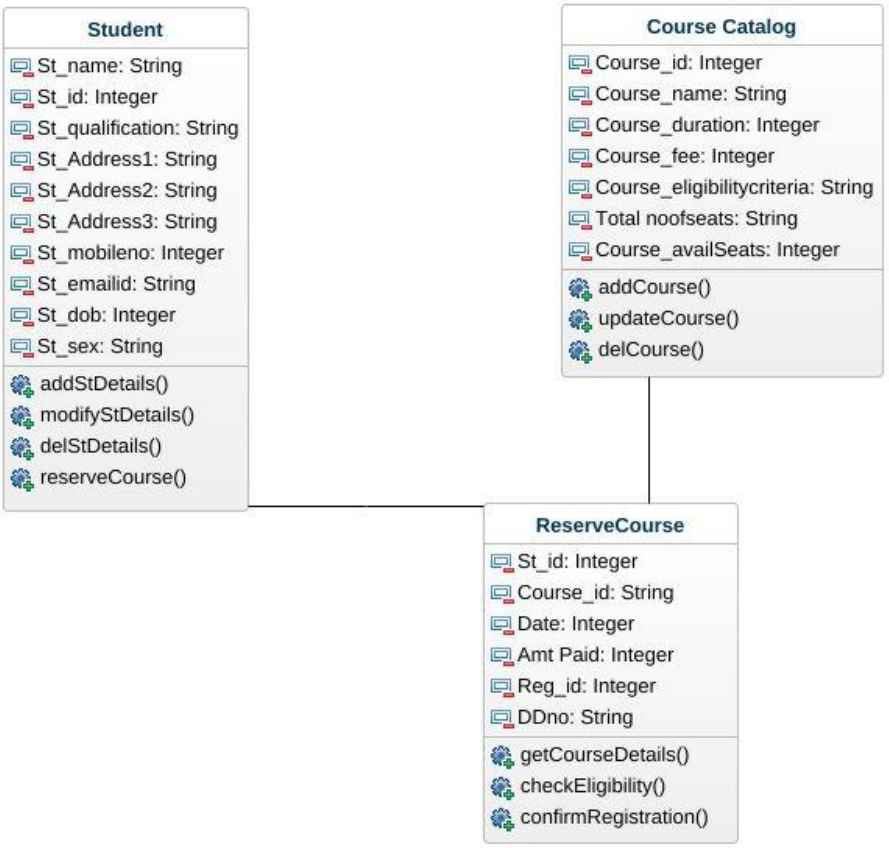
Use Case Diagram: -

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled. It is represented using stick figure.

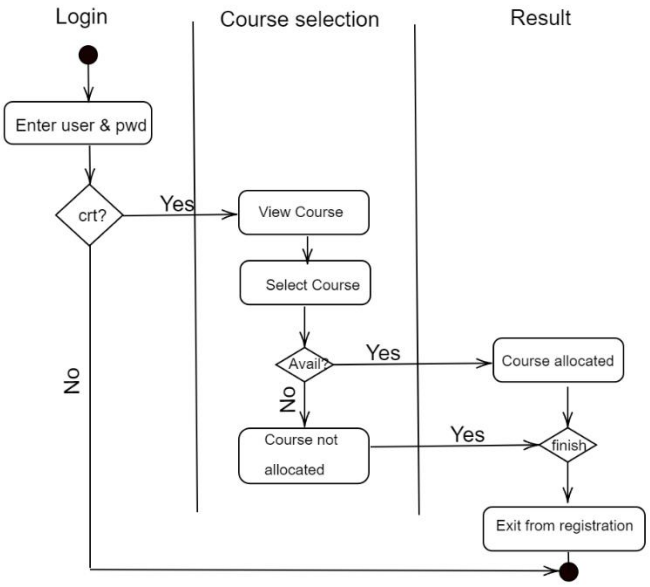


Class Diagram

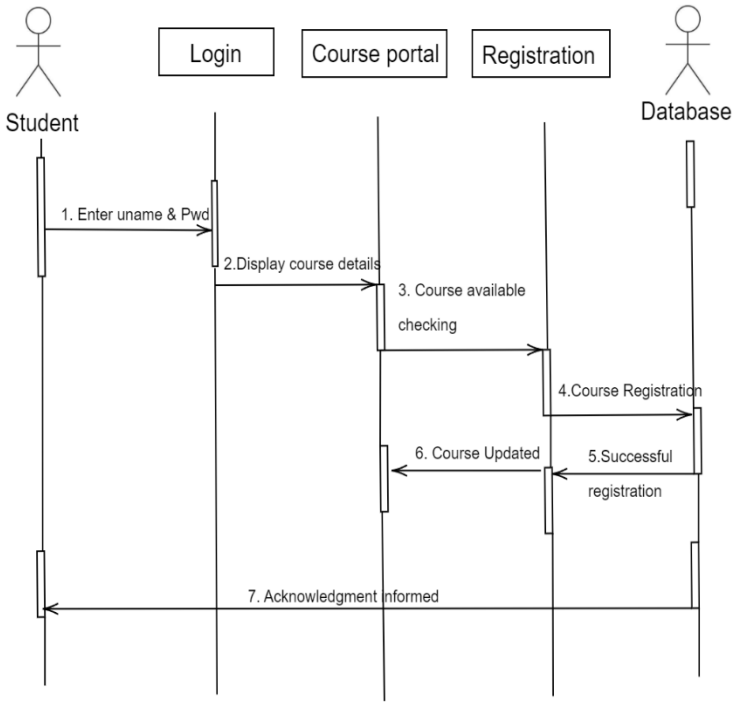
The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects



Activity diagram



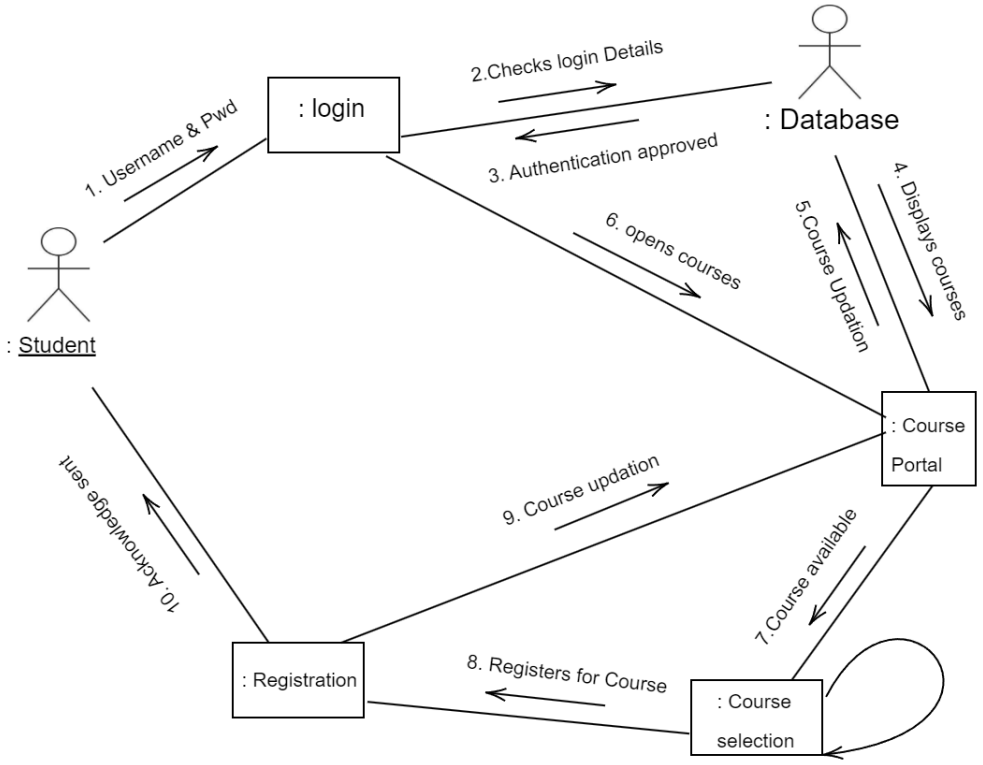
Sequence diagram



A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another, in which the “form” object is requesting an operation be performed by the “to” object. The “to” object performs the operation using a method that the class contains. It is also represented by the order in which things occur and how the objects in the system send message to one another.

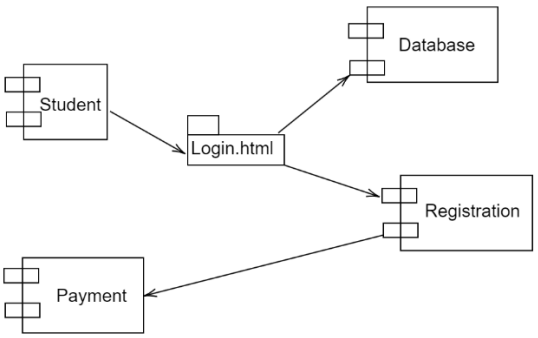
Collaboration diagram:



A collaboration diagram, also called a communication diagram or interaction diagram, A sophisticated modeling tool can easily convert a collaboration diagram into a sequence diagram and the vice versa. A collaboration diagram resembles a flowchart that portrays the roles, functionality and behavior of individual objects as well as the overall operation of the system in real time.

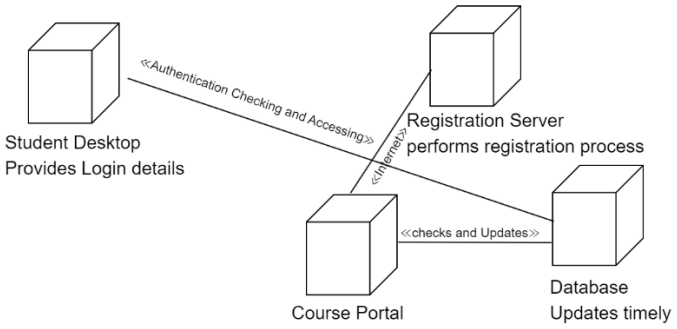
Component diagram

Component diagrams are used to visualize the organization and relationships among components in a system



Deployment diagram

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed. Component diagrams are used to visualize the organization and relationships among components in a system



RESULT:

Thus the Online Course Registration System application was successfully designed and the output was verified.

EX.NO:5	PASSPORT AUTOMATION SYSTEM

Aim:

To draw the UML diagrams [use case, class, activity, sequence, collaboration, state chart, component and deployment] for the Passport automation system.

PROBLEM STATEMENT

Passport Automation System is used in the effective dispatch of passport to all of the applicants. This system adopts a comprehensive approach to minimize the manual work and schedule resources, time in a cogent manner.

The core of the system is to get the online registration form (with details such as name, address, etc.,) filled by the applicant whose testament is verified for its genuineness by the Passport Automation System with respect to the already existing information in the database. This forms the first and foremost step in the processing of passport application.

After the first round of verification done by the system, the information is in turn forwarded to the regional administrator's (Ministry of External Affairs) office. The application is then processed manually based on the report given by the system, and any forfeiting identified can make the applicant liable to penalty as per the law. The system also provides the applicant the list of available dates for appointment to 'document verification' in the administrator's office, from which they can select one.

The system forwards the necessary details to the police for its separate verification whose report is then presented to the administrator. The administrator will be provided with an option to display the current status of application to the applicant, which they can view in online interface. After all the necessary criteria has been met, the original information is added to the database and the passport is sent to the applicant.

- **Design**
  - Use case diagram
  - Class diagram
  - Sequence diagram
  - Collaboration diagram
  - State chart diagram
  - Activity diagram
  - Component diagram
  - Deployment diagram

- **Use case**

  Identifying the Actors

  - Police
  - Applicant
  - Actor

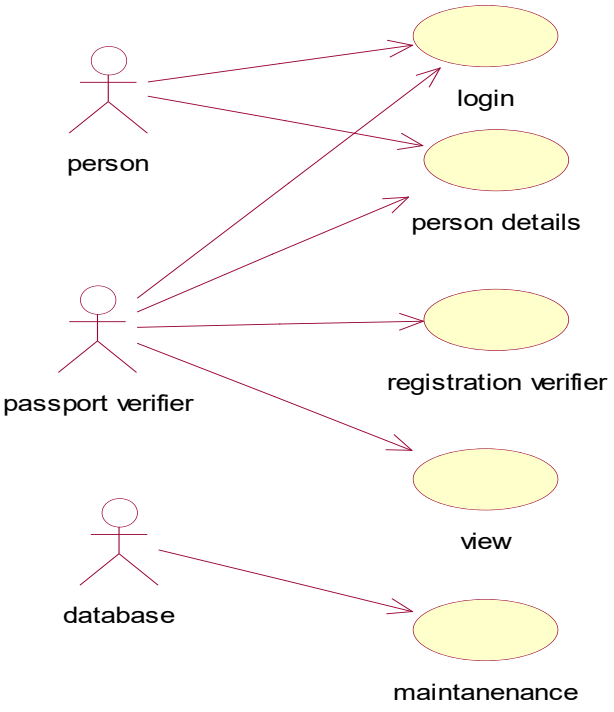
  Identifying the use case

  - Login
  - Person detail
  - View

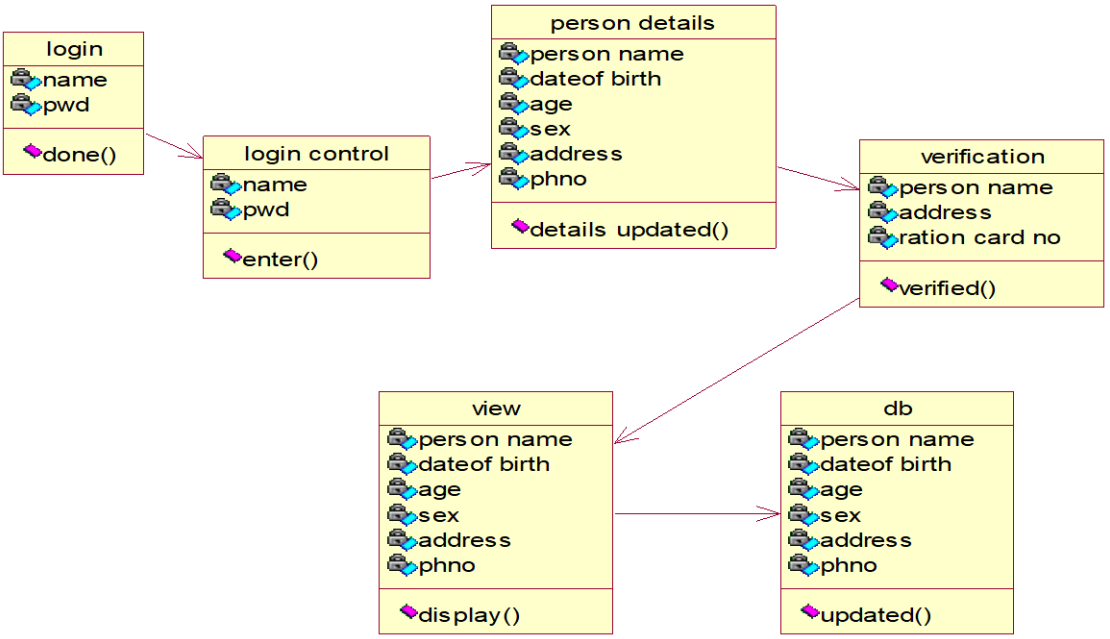
UML DIAGRAMS

Use case Diagram

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled. It is represented using stick figure.

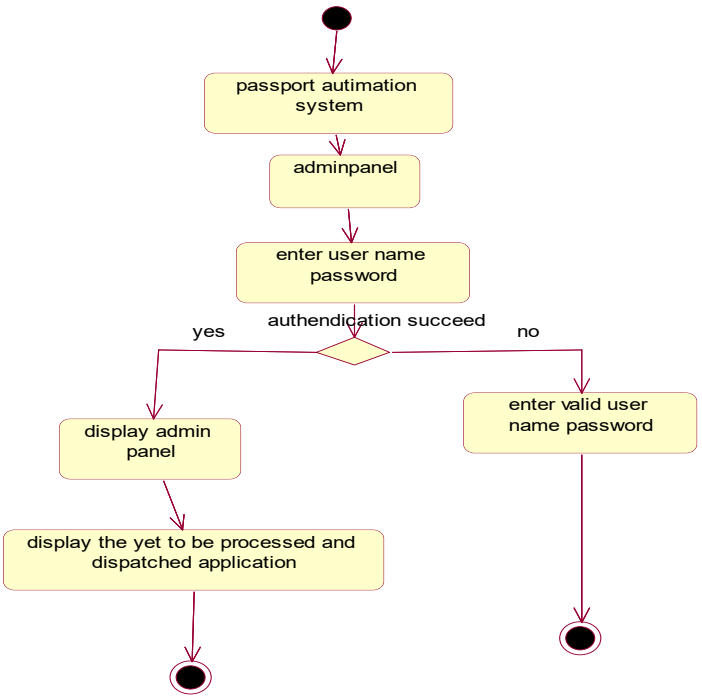


Class Diagram

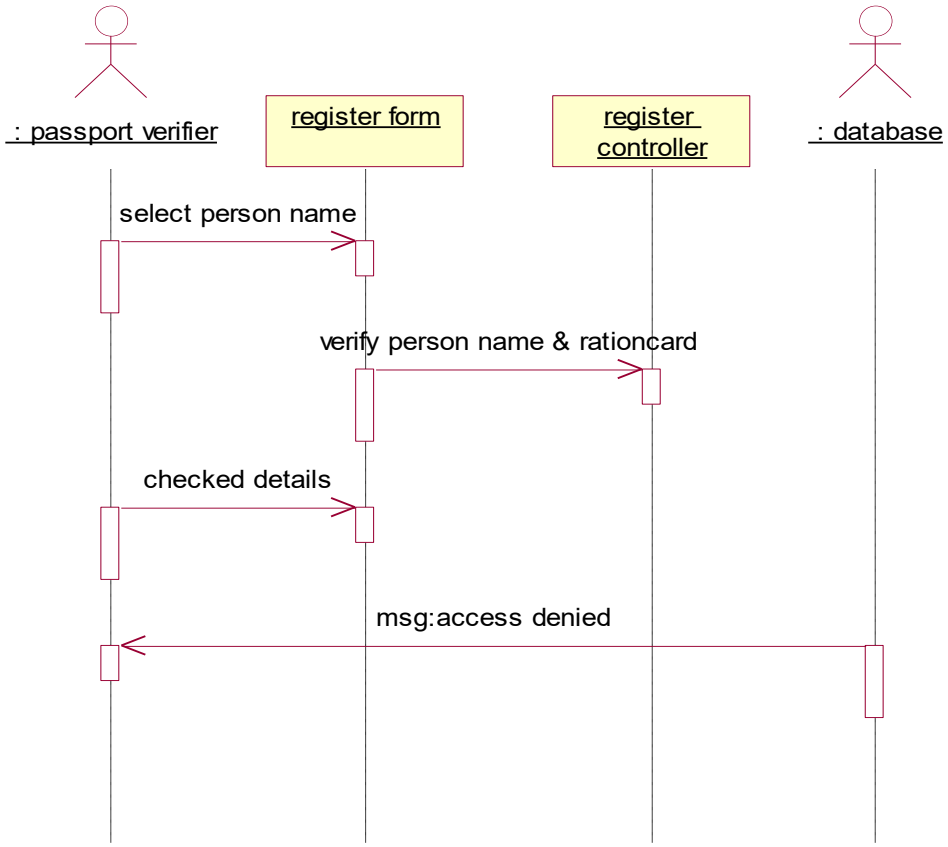


The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects

Activity Diagram



Sequence Diagram

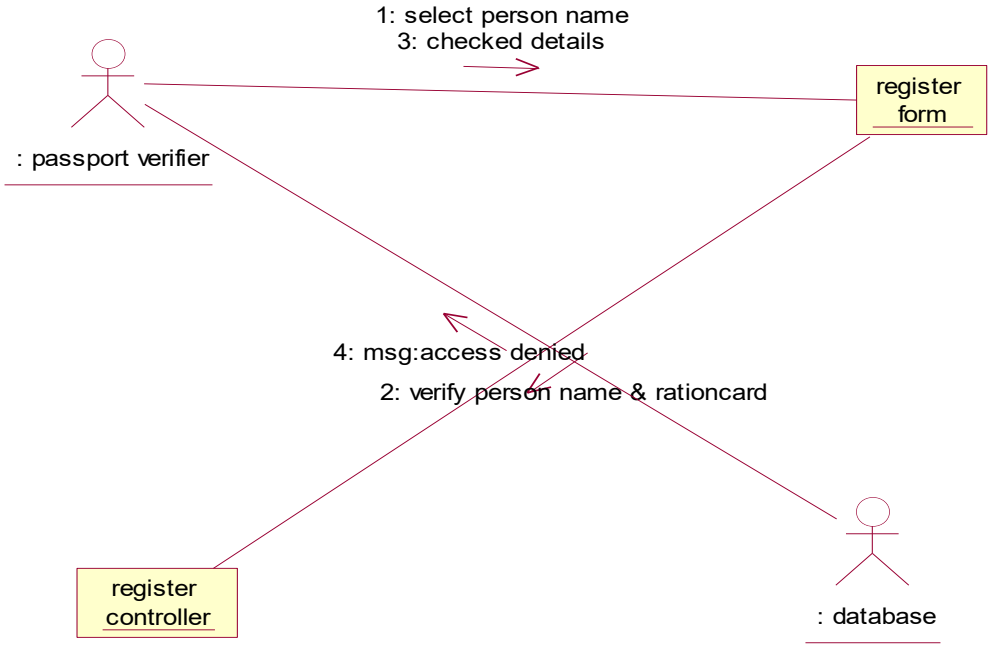


A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

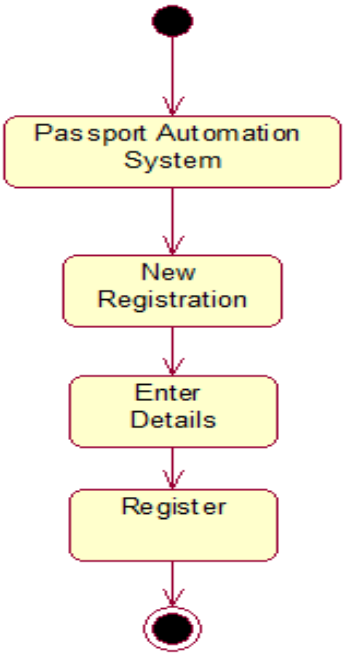
An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another, in which the “form” object is requesting an operation be performed by the “to” object. The “to” object performs the operation using a method that the class contains. It is also represented by the order in which things occur and how the objects in the system send message to one another.

Collaboration Diagram

A collaboration diagram, also called a communication diagram or interaction diagram, A sophisticated modeling tool can easily convert a collaboration diagram into a sequence diagram and the vice versa. A collaboration diagram resembles a flowchart that portrays the roles, functionality and behavior of individual objects as well as the overall operation of the system in real time.



State chart diagram



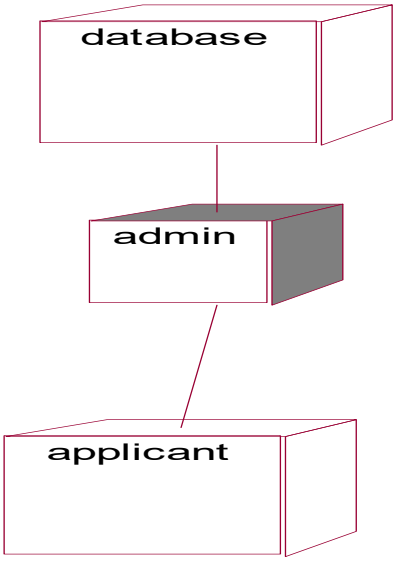
Component diagram

Component diagrams are used to visualize the organization and relationships among components in a system



Deployment Diagram

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed.



RESULT:

Thus, Passport Automation System application was successfully designed and the output was verified.

EX.NO:6	ATM MANAGEMENT SYSTEM

Aim:

To draw the UML diagrams [use case, class, activity, sequence, collaboration, state chart, component and deployment ATM for the system.

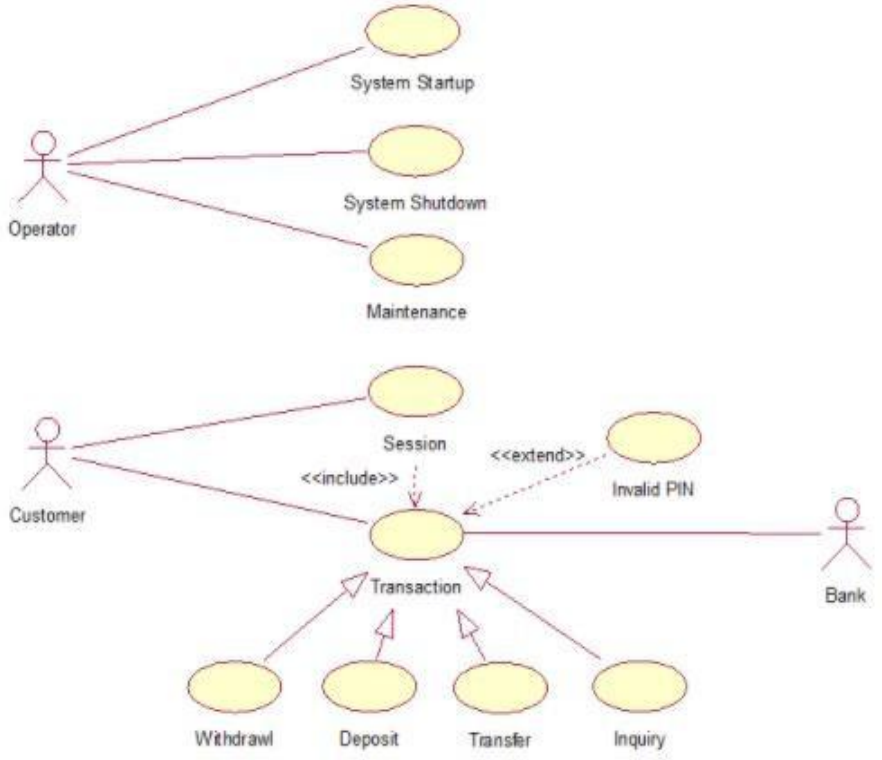
1.Problem Definition

ATM is another type of banking where the most frequently type of transaction made is withdrawal. A user may withdraw as much as many amount as he wants until his account holds a sum greater than his withdrawal amount. ATM is completely automated and there is no necessity of the ATM center being placed at the bank itself. It can be placed in the shopping malls, airports, railway stations etc.

This ATM system can use any kind of interface. But it should be user friendly and not confusing. Help manuals should be provided in case any customer has problem working with the software.

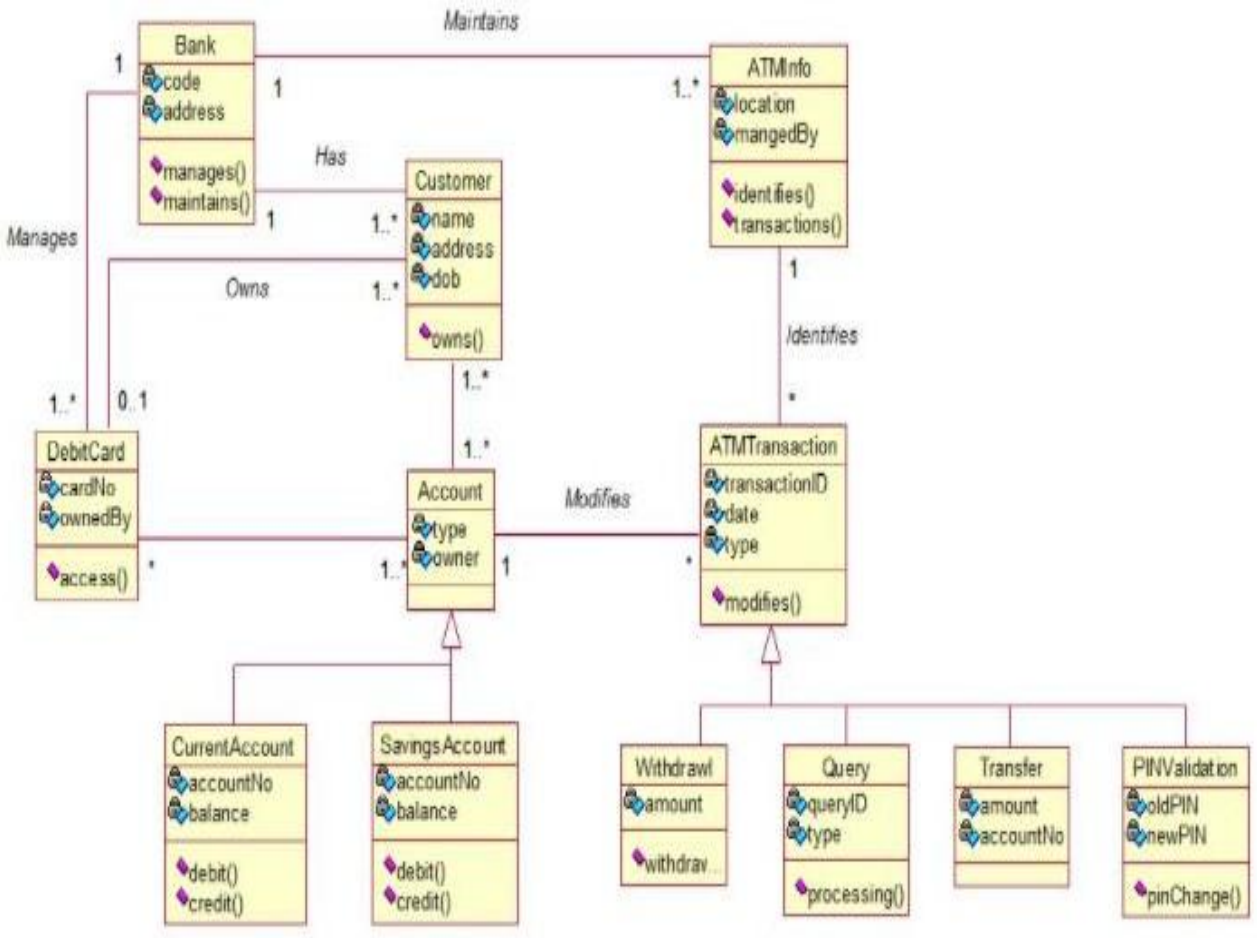
UML DIAGRAMS

USE CASE DIAGRAM



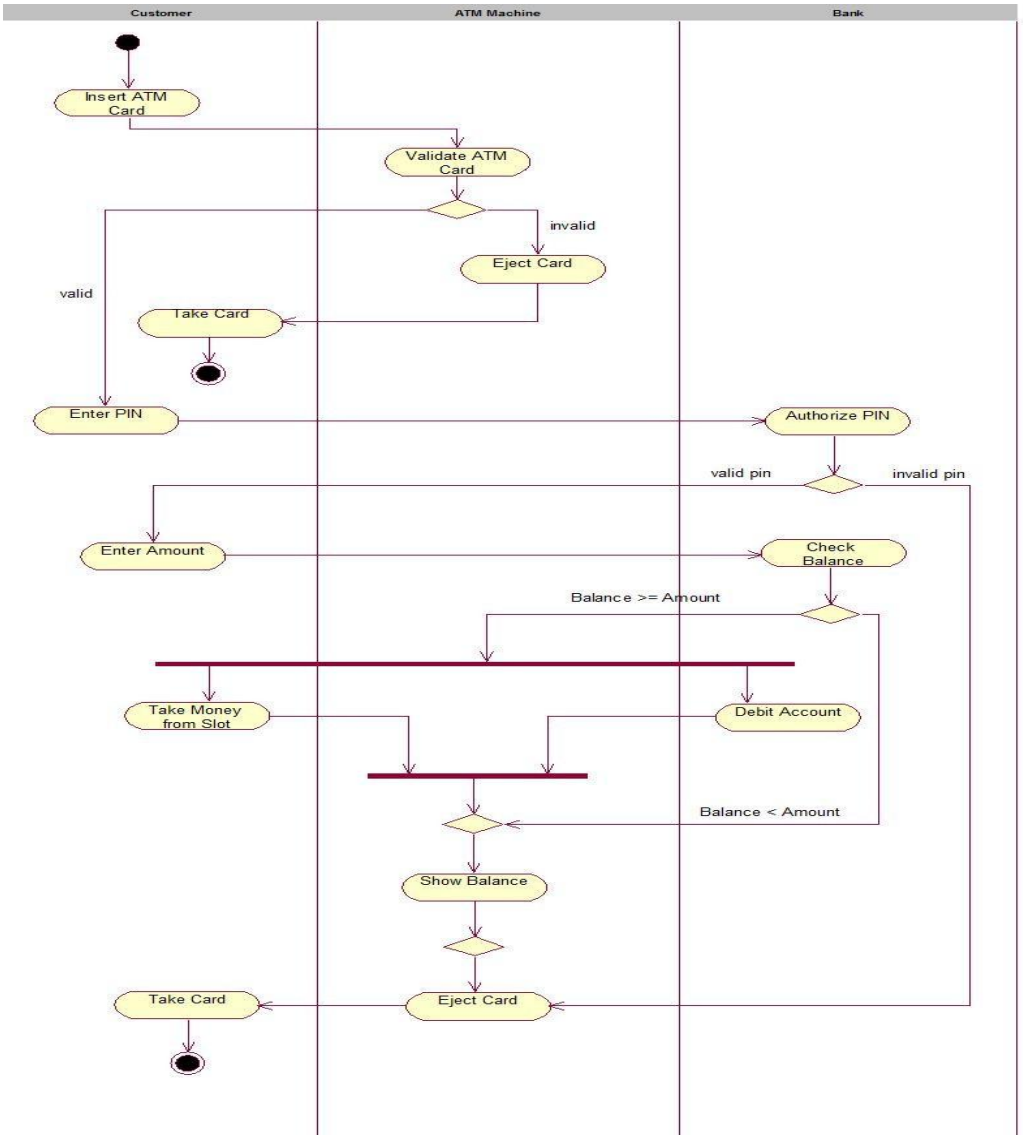
A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled

4.1 CLASS DIAGRAM

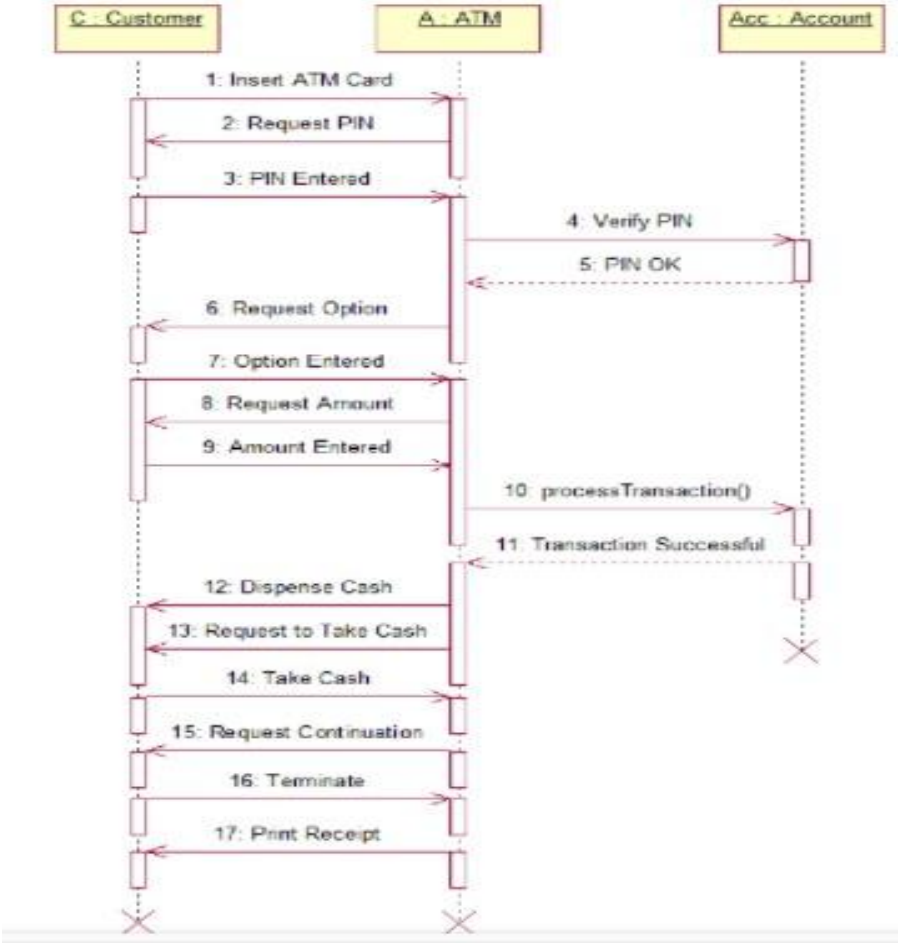


The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects.

4.2 ACTIVITY DIAGRAM



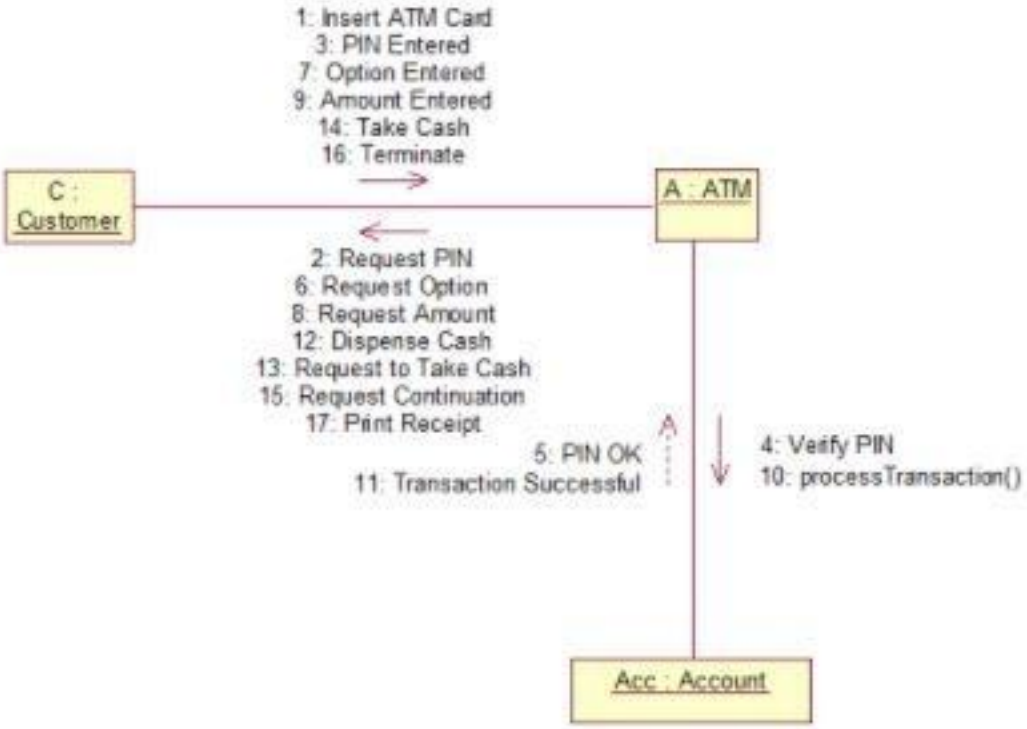
4.3 SEQUENCE DIAGRAM



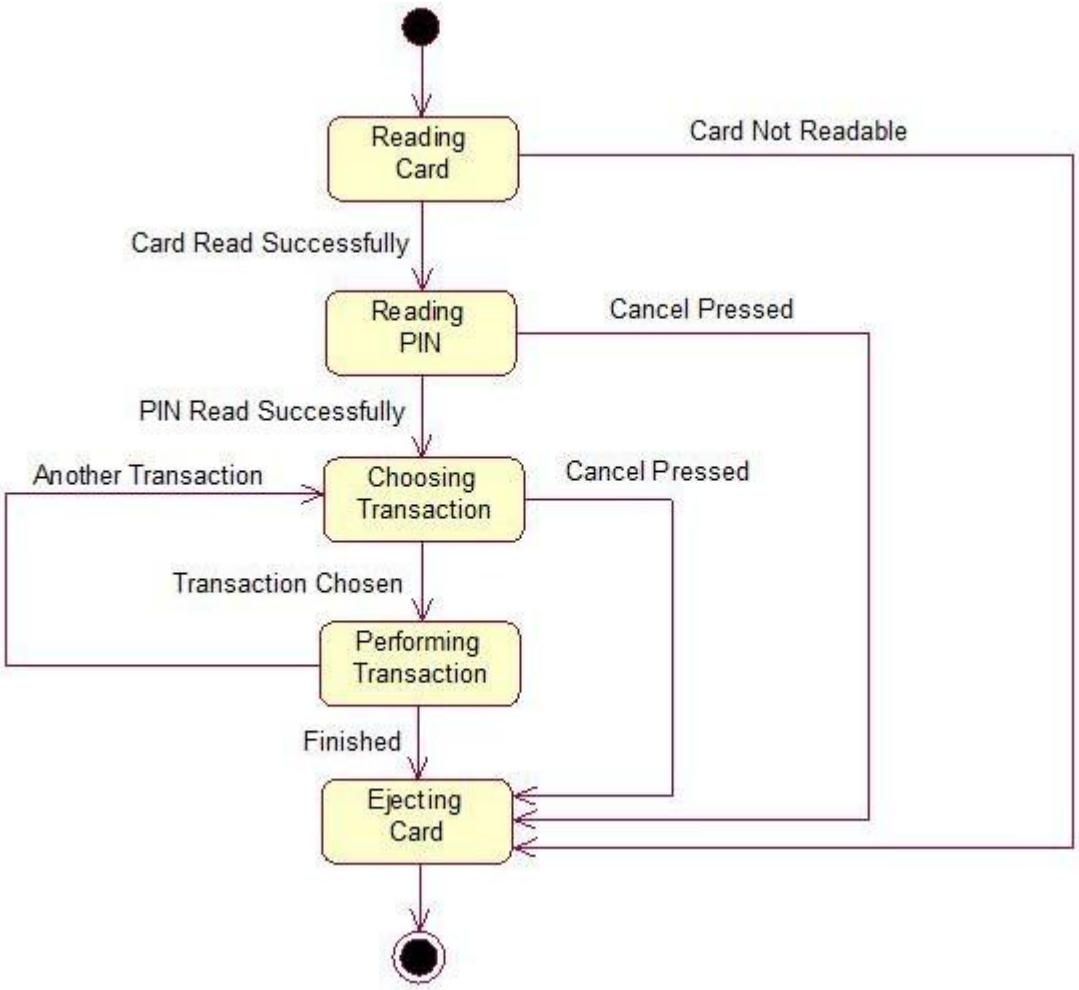
A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices. An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another, in which the “form” object is requesting an operation be performed by the “to” object. The “to” object performs the operation using a method that the class contains. It is also represented by the order in which things occur and how

the objects in the system send message to one another.

4.4 COLLABORATION DIAGRAM

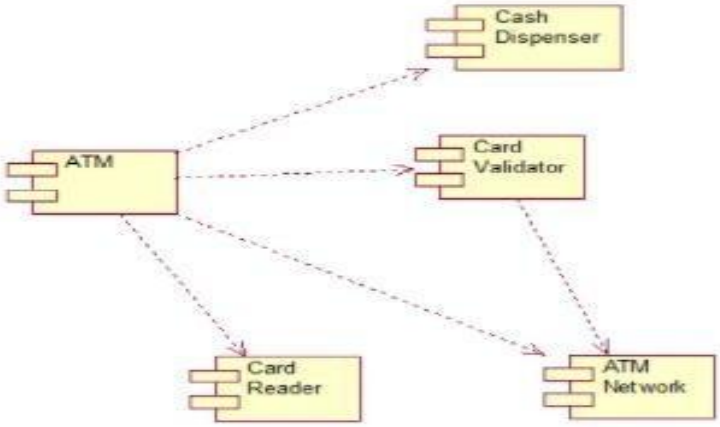


4.5 STATE CHART DIAGRAM



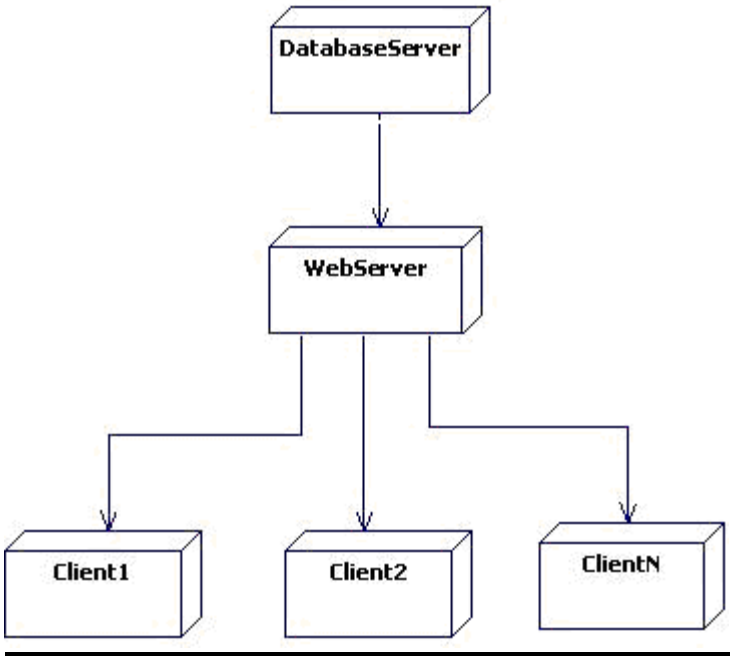
4.6 COMPONENT DIAGRAM

Component diagrams are used to visualize the organization and relationships among components in a system



4.7 DEPLOYMENT DIAGRAM

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed.



RESULT:

Thus, ATM System application was successfully designed and the output was verified.

EX.NO:7	Hospital Management System

Aim: To Develop the Problem Statement for Hospital Management System

Problem Statement

The problem statement defines the issues or challenges that the Hospital Management System intends to address. It typically includes:

Objective: To develop a software system that streamlines hospital operations, enhances patient care, and improves overall efficiency.

Challenges: Inefficient patient management, lack of centralized information, difficulty in scheduling appointments, inventory management issues, etc.

UML Diagrams

UML (Unified Modeling Language) diagrams are used to visually represent different aspects of the system. For a Hospital Management System, key UML diagrams might include:

1. Use Case Diagram

   - Illustrates interactions between users (actors) and the system

   - Includes use cases like registration, appointment scheduling, billing, etc.

2. Class Diagram

   - Shows the static structure of the system using classes and their relationships

   - Entities might include Patient, Doctor, Appointment, Billing, etc.

3. Sequence Diagram

   - Depicts the sequence of actions between objects in a particular scenario

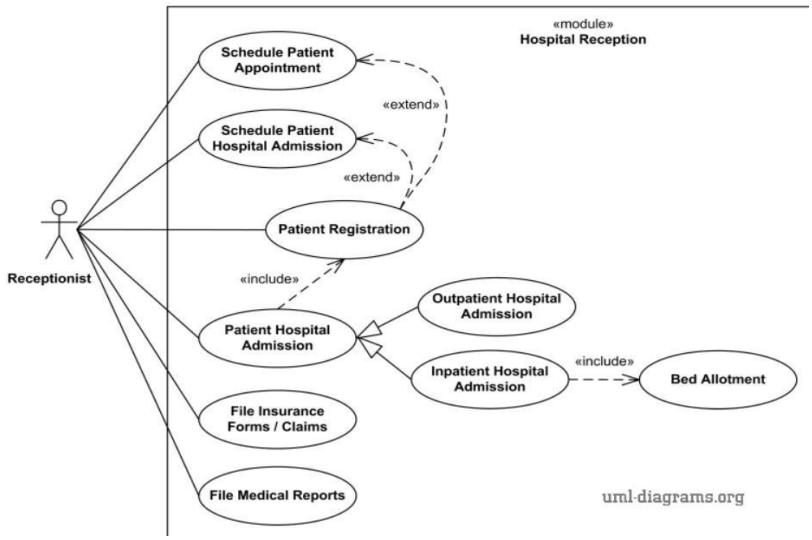
   - Useful for showing interactions during processes like appointment scheduling or billing

4. Activity Diagram

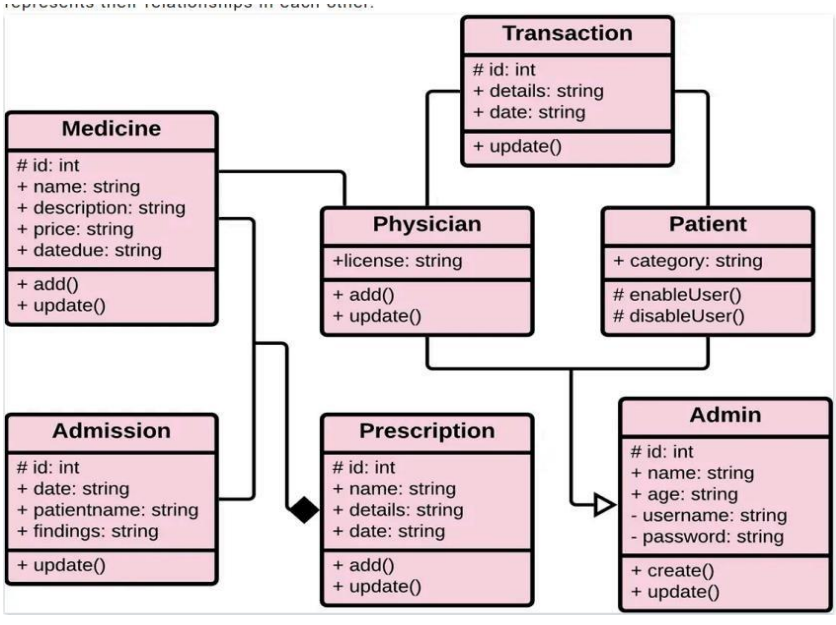
- Describes the flow of activities or processes within the system
- Can illustrate processes such as patient admission, discharge, or treatment flow

Use case Diagram:

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled. It is represented using stick figure.

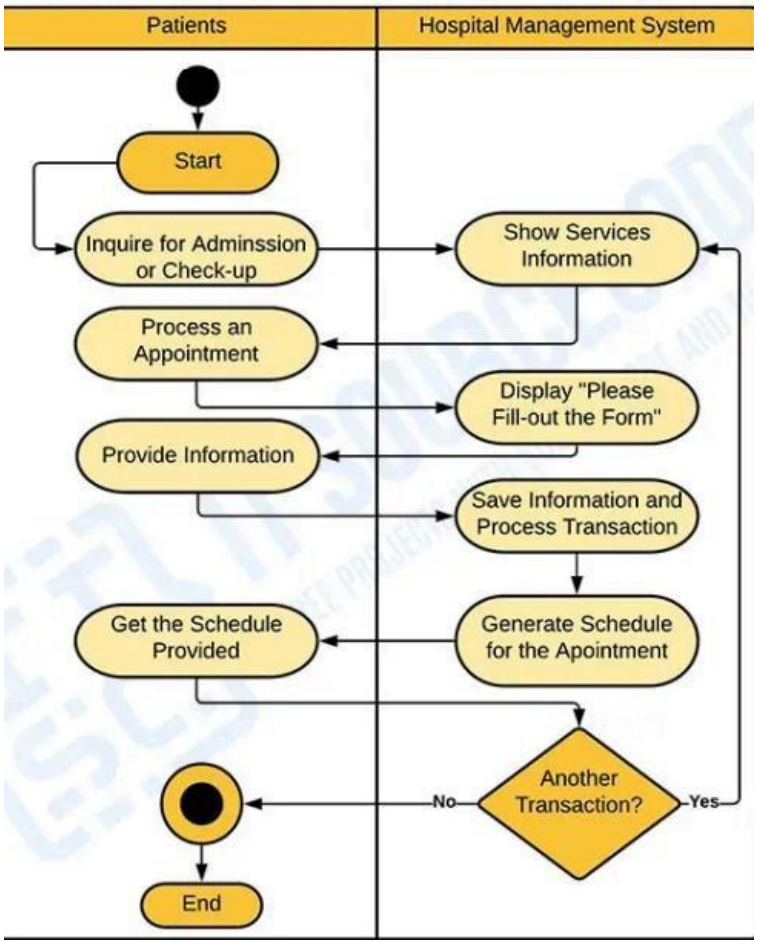


Class Diagram

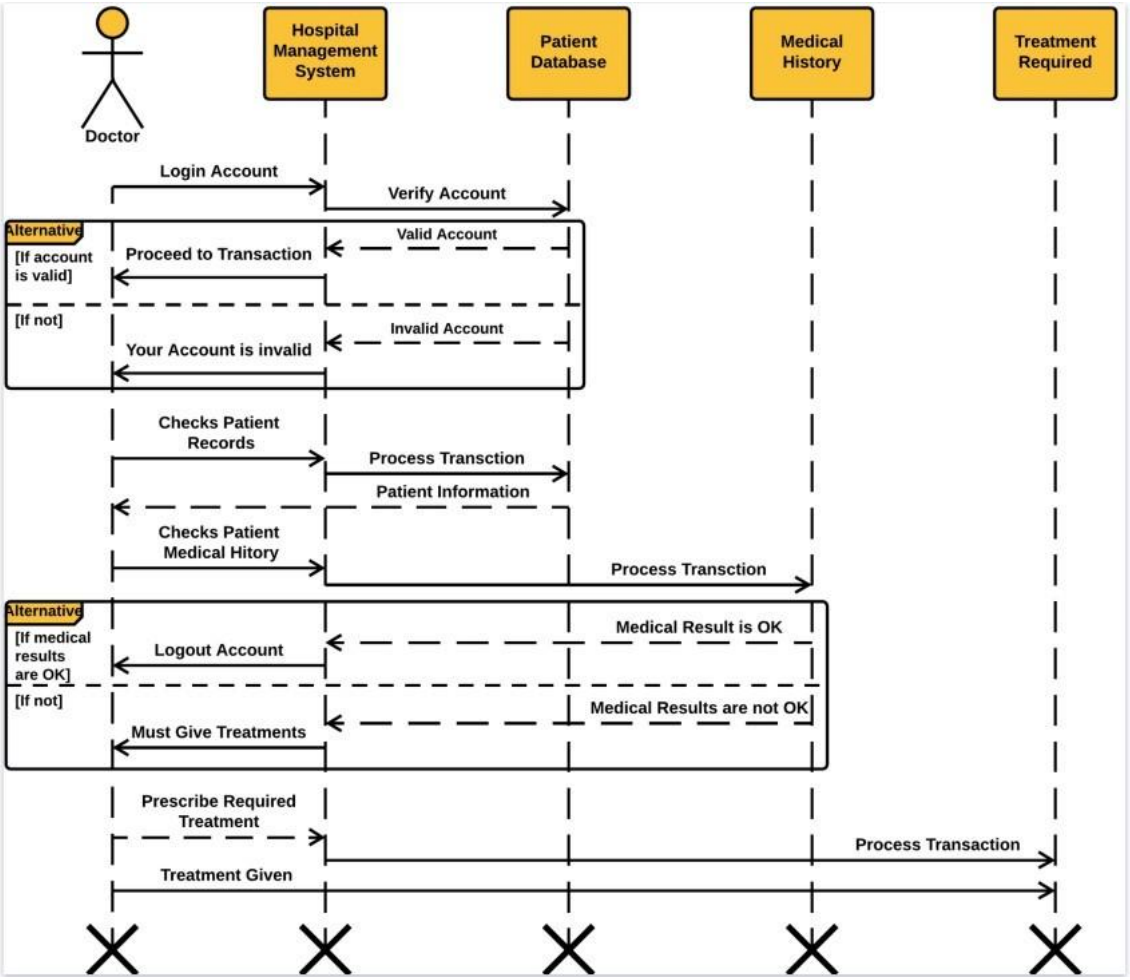


The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects.

Activity Diagram



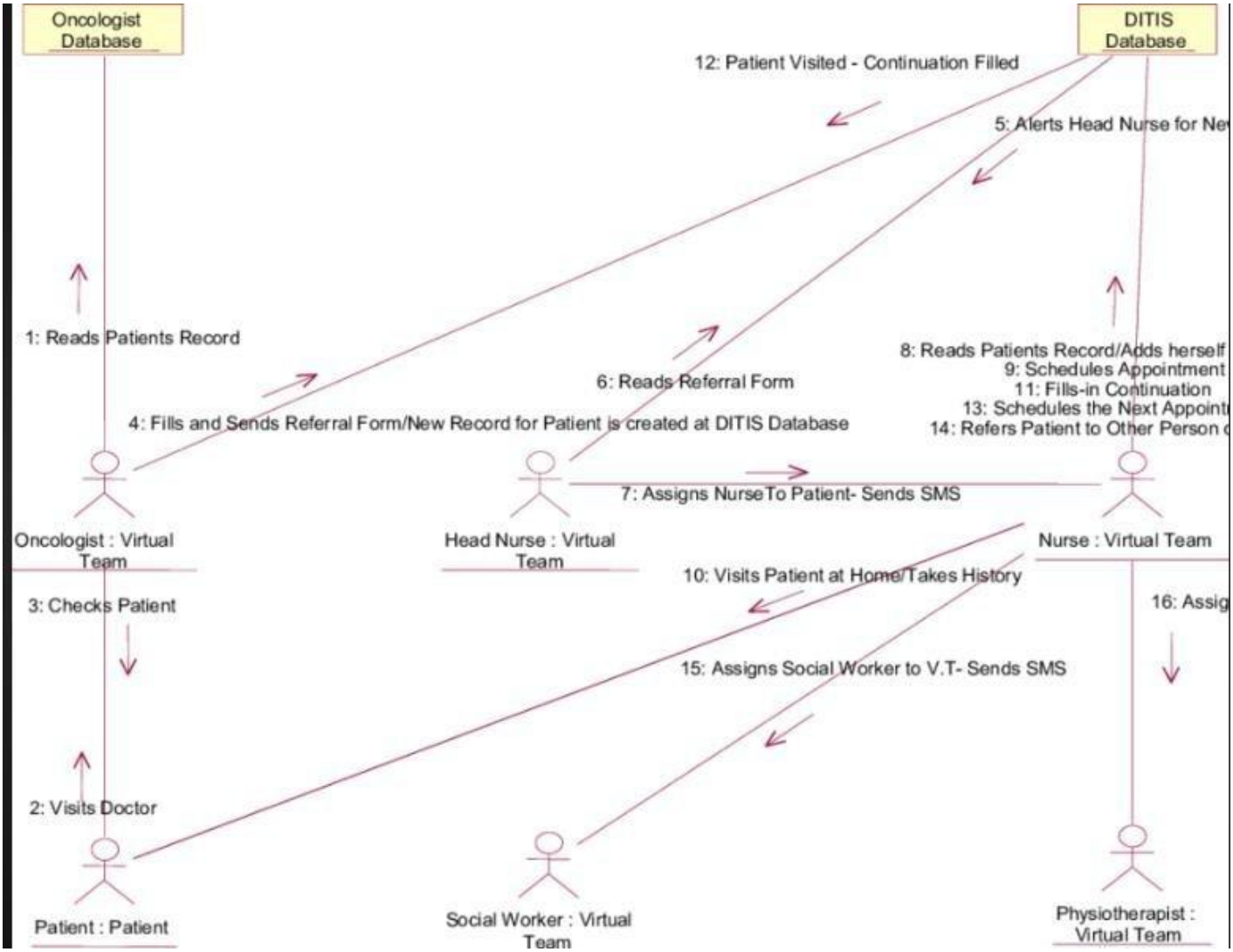
Sequence Diagram



A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another, in which the “form” object is requesting an operation be performed by the “to” object. The “to” object performs the operation using a method that the class contains. It is also represented by the order in which things occur and how the objects in the system send message to one another.

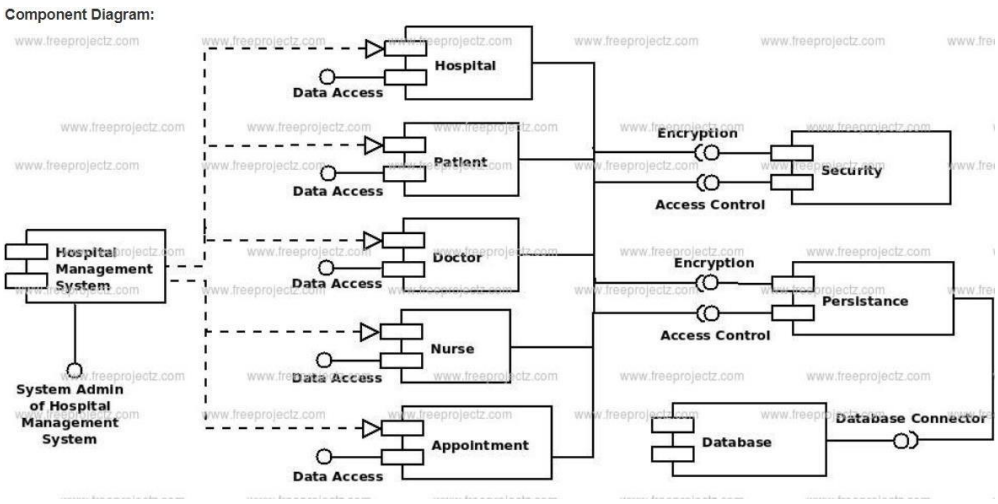
Collaboration Diagram



A collaboration diagram, also called a communication diagram or interaction diagram, A sophisticated modeling tool can easily convert a collaboration diagram into a sequence diagram and the vice versa. A collaboration diagram resembles a flowchart that portrays the roles, functionality and behavior of individual objects as well as the overall operation of the system in real time.

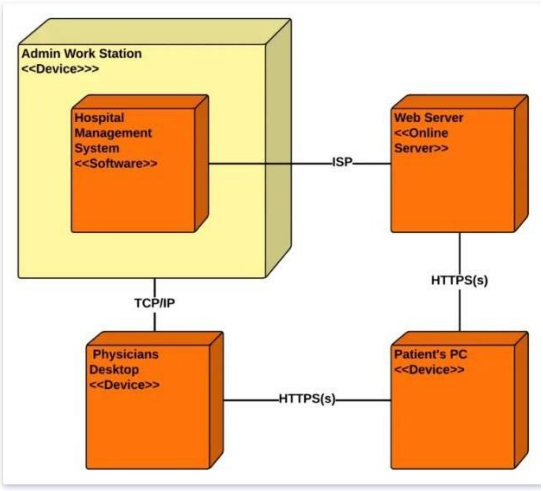
Component Diagram

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed. Component diagrams are used to visualize the organization and relationships among components in a system



Deployment Diagram

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed.



RESULT

Thus, the Hospital management system has been successfully executed and codes are generated.

Ex. No: 8 CONFERENCE MANAGEMENT SYSTEM

AIM

To develop a project on Conference management system using Rational Rose Software.

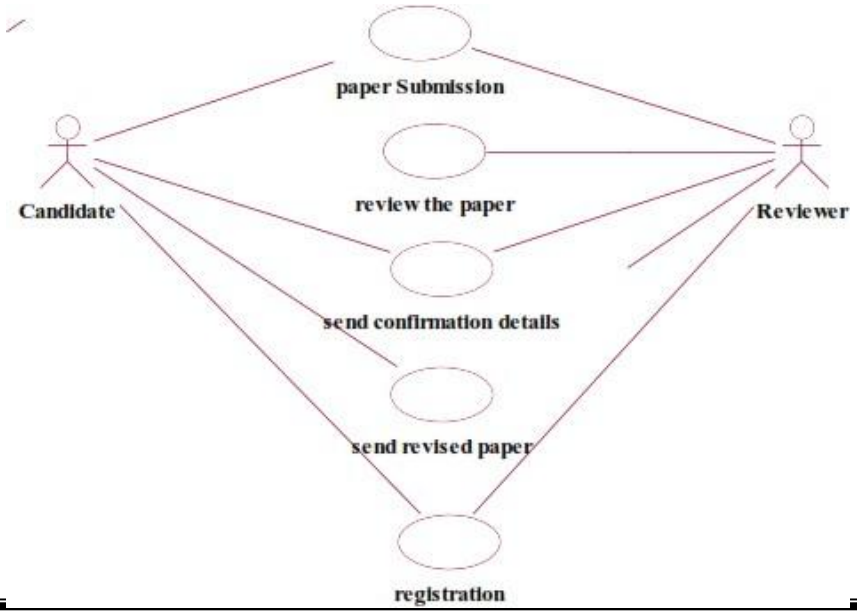
PROBLEM STATEMENT

The process of the candidates is to login the conference system and submit the paper through online. Then the reviewer reviews the paper and sends the acknowledgement to the candidate either paper selected or rejected. This process of on conference management system are described sequentially through following steps,

- The candidate login to the conference management system.
- The paper title is submitted.
- The paper is been reviewed by the reviewer.
- The reviewer sends acknowledgement to the candidate.
- Based on the selection, the best candidate is selected.
- Finally, the candidate registers all details.

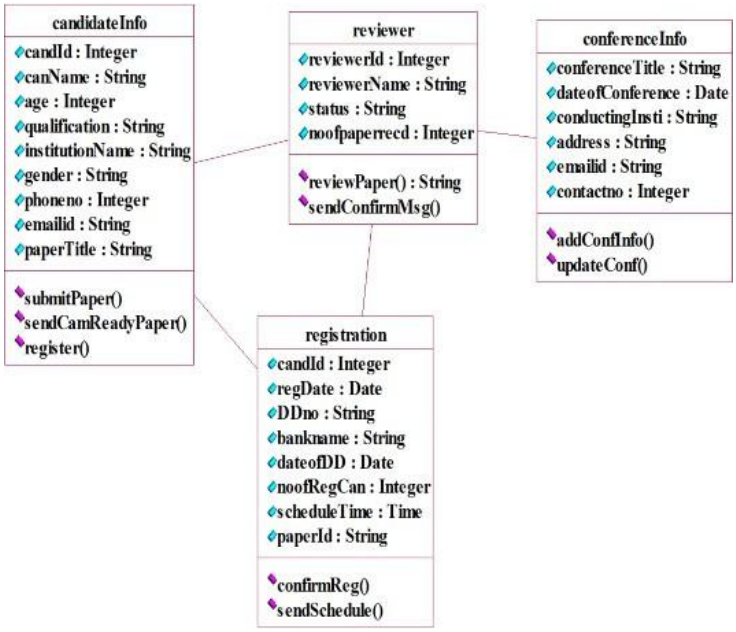
USE CASE DIAGRAM:

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It is represented using ellipse. Actor is any external entity that makes use of the system being modeled.



CLASS DIAGRAM

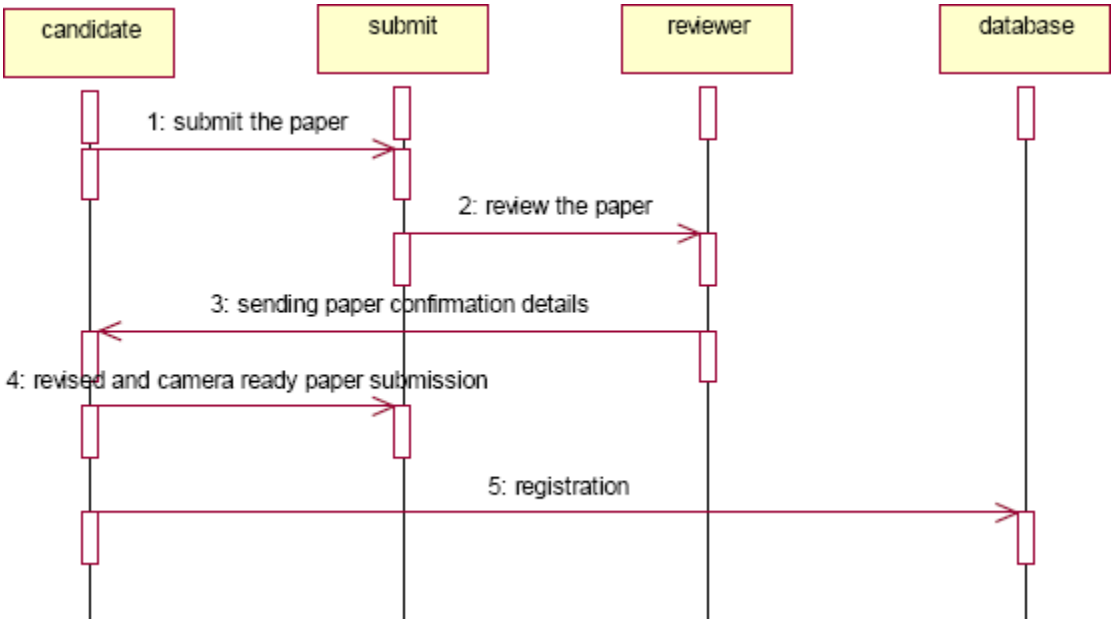
A class diagram in the unified modeling language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, and the relationships between the classes. It is represented using a rectangle with three compartments. Top compartment has the class name, middle compartment the attributes and the bottom compartment with operations.



SEQUENCE DIAGRAM

A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

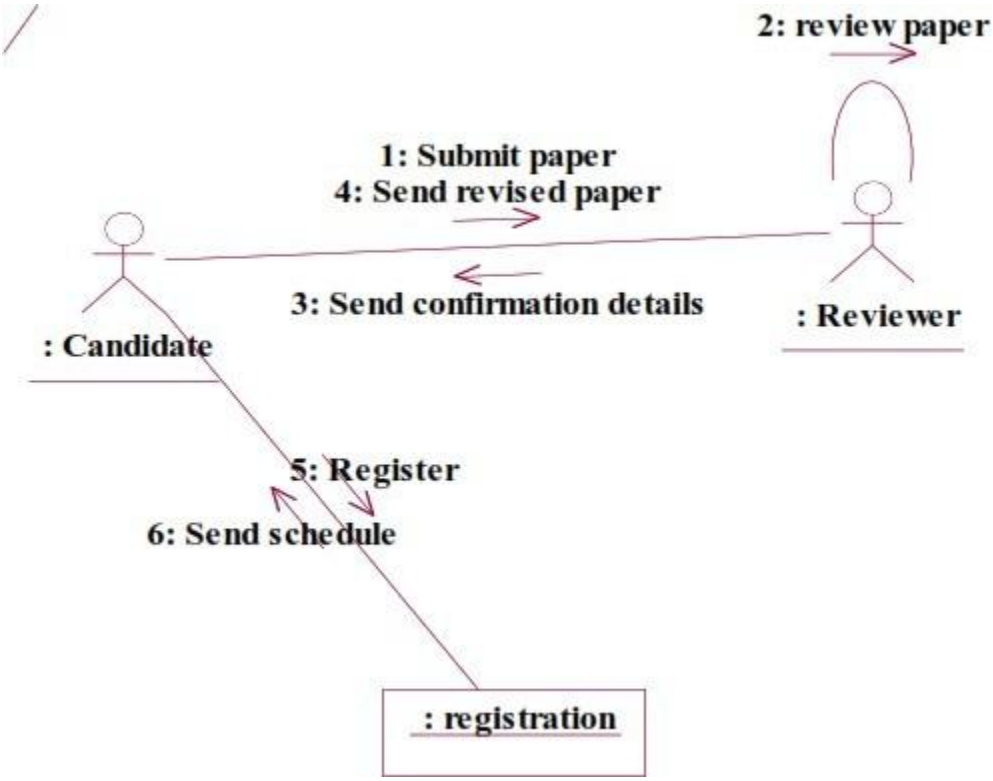
An event also is considered to be any action by an object that sends information. The event line represents a message sent from one object to another, in which the “form” object is requesting an operation be performed by the “to” object. The “to” object performs the operation using a method that the class contains. It is also represented by the order in which things occur and how the objects in the system send message to one another.



collaboration diagram

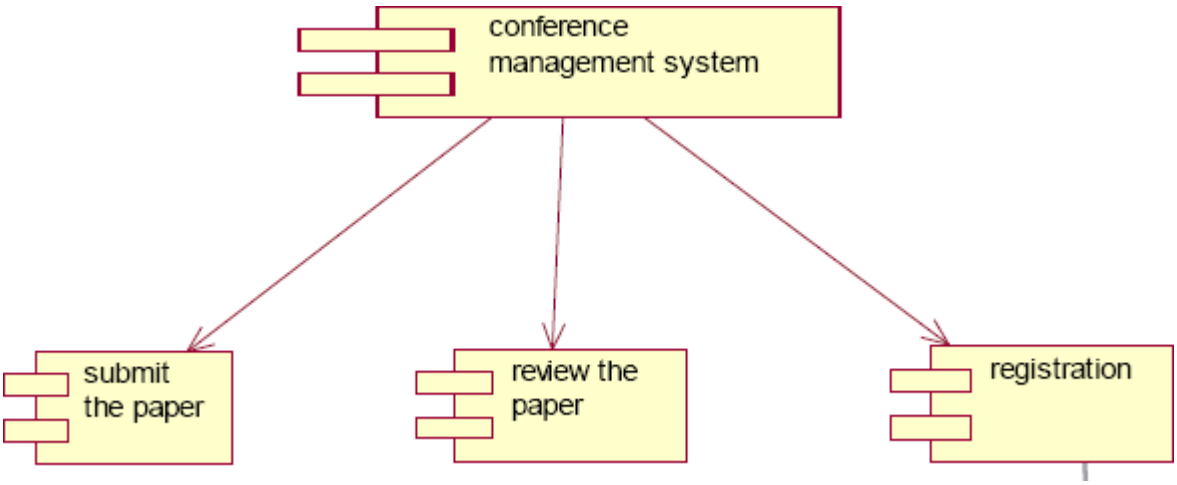
A collaboration diagram,

also called a communication diagram or interaction diagram, A sophisticated modeling tool can easily convert a collaboration diagram into a sequence diagram and the vice versa. A collaboration diagram resembles a flowchart that portrays the roles, functionality and behavior of individual objects as well as the overall operation of the system in real time.



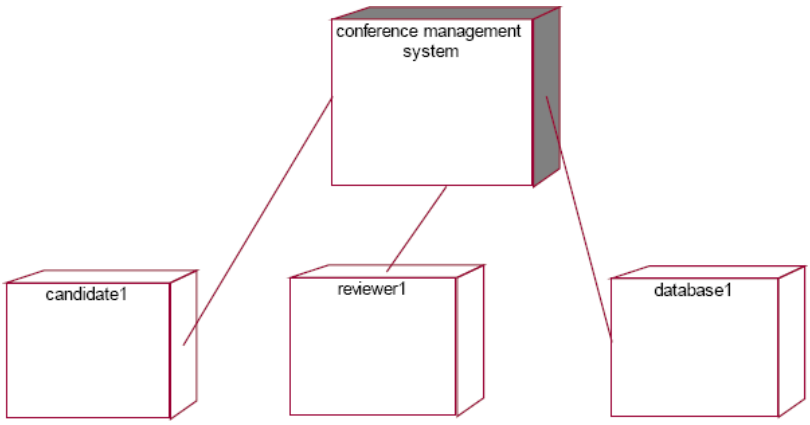
Component diagram

The component diagram's main purpose is to show the structural relationships between the components of a system. It is represented by boxed figure. Dependencies are represented by communication association.



DEPLOYMENT DIAGRAM

A deployment diagram in the unified modeling language serves to model the physical deployment of artifacts on deployment targets. Deployment diagrams show "the allocation of artifacts to nodes according to the Deployments defined between them. It is represented by 3-dimensional box. Dependencies are represented by communication association.



RESULT

Thus, the Conference management system has been successfully executed and codes are generated.